

New Quantum Circuits for ECDLP: Breaking Prime Elliptic Curve Cryptography

Hyunji Kim^{1*}, Kyungbae Jang^{2*}, Siyi Wang², Vikas Srivastava³, Anubhab Baksi⁴, Gyeongju Song¹, Hwajeong Seo¹, and Anupam Chattopadhyay^{2†}

¹ Division of IT Convergence Engineering, Hansung University, Seoul, South Korea

² College of Computing and Data Science, Nanyang Technological University, Singapore

³ Department of Mathematics, National Institute of Technology Warangal, India

⁴ Elektro- och informationsteknik, Lund University, Lund, Sweden

khj1594012@gmail.com, starj1023@gmail.com, siyi002@e.ntu.edu.sg,
vikas.math123@gmail.com, anubhab.baksi@eit.lth.se, thdrudwn98@gmail.com,
hwajeong84@gmail.com, anupam@ntu.edu.sg

Abstract. This paper improves quantum circuits for realizing Shor’s algorithm on elliptic curves. We present optimized quantum point addition circuits that focus on reducing circuit depth at the cost of using more qubits. Our implementation includes in-place and out-of-place point additions, considering both affine and projective coordinates, respectively. This significantly reduces the circuit depth and achieves about 58%–82% improvement in the qubit count – T -depth product and 43%–87% improvement in the qubit count – full depth product over previous works, including those of M. Roetteler et al. (Asiacrypt 2017) and T. Häner et al. (PQCrypto 2020).

Based on these circuits, we construct Shor’s algorithm and evaluate the post-quantum security of elliptic curve cryptography. Under the MAXDEPTH constraint proposed by NIST, which limits the maximum circuit depth to 2^{40} , the maximum depth in our work is $2^{28.9}$ for the P-521 curve. For the total gate count and full depth product, a metric defined by NIST for evaluating quantum attack resistance, the maximum complexity for the same curve is $2^{65.6}$, far below the post-quantum security level 1 requirement of 2^{157} .

Beyond these logical analyses, we estimate the fault-tolerant costs (i.e., at the level of physical resources) for breaking elliptic curve cryptography. As one of our results, the P-224 curve (comparable to RSA-2048 in classical security) can be broken in 1.4 minutes using about 107 million physical qubits, or in 14.3 hours using about 670000 physical qubits.

Keywords: Shor’s Algorithm · Elliptic Curves · Quantum Cryptanalysis

*: These authors contributed equally to this work.

†: Corresponding author.

1 Introduction

Elliptic Curve Cryptography (ECC) [Kob87] stands as a cornerstone of modern cryptography, widely adopted for its ability to provide robust security with smaller key sizes than RSA [RSA78]. Its arithmetic structure over finite fields enables efficient implementations for key exchange, digital signatures, and other standardized protocols. Widely deployed elliptic curve standards comprise the NIST P-curves (e.g., P-256 and P-521), Curve25519, and the Brainpool curves, each targeting a distinct level of cryptographic strength.

The security of ECC relies on the computational hardness of the elliptic curve discrete logarithm problem (ECDLP). However, the advent of large-scale quantum computers undermines this hardness, as Shor’s algorithm [Sho94] can solve the ECDLP in polynomial time, thereby breaking the security of elliptic curves. The same quantum threat also applies to RSA, whose security is based on the hardness of integer factorization. As a result of this quantum threat, the research community has been driven to seek quantum resistant alternatives to existing public key cryptosystems, giving rise to the field now known as post-quantum cryptography (see the NIST competition⁵).

Alongside the development of post-quantum cryptography, it remains important to assess the security of deployed public key cryptographic systems. From a quantum cryptanalysis perspective, it is natural to ask how costly it would be to break ECC. Accordingly, several studies have been conducted to evaluate the quantum resources required for breaking prime field ECC using Shor’s algorithm. Proos and Zalka [PZ03] presented the first detailed analysis of Shor’s algorithm applied to elliptic curves. Building upon this groundwork, Roetteler et al. [RNSL17] presented a comprehensive implementation of Shor’s algorithm for ECC, establishing a solid foundation for subsequent quantum resource analyses. Following this, Häner et al. [HJN⁺20] significantly improved efficiency through refined quantum circuit designs and further analyzed trade-offs across various metrics. Based on the quantum circuits from [HJN⁺20], Gouzien et al. [GRLR⁺23] presented physical-level optimizations for breaking elliptic curves. In Eurocrypt 2026 [CFS26], the authors used projective coordinates to achieve the lowest qubit count reported so far, at the cost of a higher gate count. Around the same time, Luo et al. [LYW⁺26] achieved a slightly higher qubit count but a lower Toffoli gate count than [CFS26]. Very recently, Babbush et al. [BZG⁺26] presented an advanced quantum attack on the elliptic curve secp256k1 [Bro10], which is used in blockchain systems, using a small number of qubits and Toffoli gates (while circuit depth is not reported). Note that the details of their quantum circuits are not publicly available due to responsible disclosure. As a side note, studies on binary field ECC [BBvHL20, TT23, JSB⁺25] and Ed25519 [HZH⁺25] have also been conducted.

In this paper, we contribute to this body of work by presenting new quantum circuits for breaking ECC. We estimate the logical resources required to break

⁵ <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/call-for-proposals>

ECC, including quantum gates, the number of qubits, and circuit depth (the number of sequential gate layers). Furthermore, we analyze physical costs such as runtime and the number of physical qubits required for error correction under fault-tolerant computation. Source code for our quantum circuits is provided in the supplementary material and will be made available online in the final version.

Contribution

This paper presents new quantum circuits for elliptic curve computations, prioritizing low circuit depth through the additional use of qubits. We propose two types of quantum point addition circuits (the main cost contributor in Shor’s algorithm): *in-place* and *out-of-place* versions. The in-place point addition targets *affine* coordinates and does not produce garbage qubits. On the other hand, the out-of-place point addition considers *projective* coordinates and produces garbage qubits. The latter is more efficient in terms of quantum gate count and circuit depth, and two approaches are considered to mitigate the overhead of these garbage qubits. Our implementation includes the following optimizations at the component and architecture levels:

- We present depth- and qubit-optimized modular multiplications (Section 3.2) based on the Montgomery algorithm. For the depth-optimized version, we employ a *Quantum Carry-Save Adder* (QCSA) [KLJ⁺25] and reformulate the Montgomery reduction into an alternative form that enables the use of the QCSA. The qubit-optimized version reduces the number of qubits by lowering the degree of parallelism in the QCSA.
- We develop a quantum inversion circuit (Section 3.3) based on the binary Extended Euclidean Algorithm (EEA). Our inversion circuit introduces a low-depth round operation with unconditional execution and a postponed modular reduction strategy. Since the postponed reduction accumulates multiple additions, we employ the QCSA to handle them efficiently.
- The in-place point addition (Section 4.1) is implemented in affine coordinates. Our architecture builds on that of [HJN⁺20] with two modifications and is further optimized by employing the multiplication and division methods developed in this work. The out-of-place point addition targets projective coordinates, requires 11 multiplications, and no division, but inherently produces garbage qubits. We fully utilize our QCSA-based multiplication, and consider two methods to mitigate these garbage qubits: *windowing* and the *zig-zag* architecture.

Overall, our implementation uses a high number of qubits and provides low quantum gate count and circuit depth. To evaluate performance, we adopt the trade-off metric given by the product of qubit count and depth, which is often used in quantum implementations [JNRV20,ZWS⁺20,LPZW23,LL25,JSB⁺25]. We denote the qubit count – T -depth product by TD - M and the qubit count – full depth product by FD - M . Compared to the previous best result by Häner et al. [HJN⁺20] (in terms of this metric), our point addition achieves 58%–82%

and 43%–87% improvements⁶ in TD - M and FD - M costs, respectively. Further, we reduce the total gate count by 61%–93%.

By constructing Shor’s attack using our point addition circuits, we estimate the required quantum resources for breaking elliptic curve cryptography (Section 6.1). Unsurprisingly, ECC does not satisfy NIST post-quantum security [NIS22, pp. 15–17] under our analysis (see Table 8) and can be attacked with significantly lower depth than the NIST MAXDEPTH [NIS22, p. 17]. Based on these logical-level results, we provide more realistic fault-tolerant cost estimates by computing the required number of physical qubits and the actual runtime (Section 6.2) under surface codes [FMMC12] and quantum low-density parity check (QLDPC) codes [WSC25]. Tables 6 and 7 report these costs across various target curves, hardware assumptions, and optimization strategies.

2 Background

2.1 Prime Fields $\text{GF}(p)$

Let p be a prime number. The prime field $\text{GF}(p)$ is defined as the finite field containing exactly p elements, namely $\text{GF}(p) = \{0, 1, 2, \dots, p-1\}$. The elements of $\text{GF}(p)$ are integers modulo p , and the field arithmetic is defined in terms of modular arithmetic. In particular, for all $a, b \in \text{GF}(p)$, the operations are given by $a + b \equiv (a + b) \pmod{p}$, $a \cdot b \equiv ab \pmod{p}$. Since p is prime, every nonzero element $a \in \text{GF}(p)$ has a multiplicative inverse a^{-1} such that $a \cdot a^{-1} \equiv 1 \pmod{p}$, and hence $\text{GF}(p)$ satisfies all the field axioms. Examples of prime fields can be found in Examples 1 (toy), 2 (standard), and 3 (standard).

2.2 Prime Elliptic Curves

Let p be a prime and let $\text{GF}(p)$ denote the corresponding prime field. An elliptic curve defined over $\text{GF}(p)$ is called an elliptic curve over a prime field, or a *prime elliptic curve*. The points on such a curve are pairs (x, y) with $x, y \in \text{GF}(p)$ satisfying a defining equation, together with a special point $\mathcal{O}$ (the point at infinity). This set forms an abelian group under a suitable addition law. According to standard formulations (e.g., NIST), elliptic curves over prime fields can be represented in three main forms:

(i) Short Weierstrass Form. An elliptic curve in short Weierstrass form is defined by

$$W_{a,b} : y^2 \equiv x^3 + ax + b \pmod{p},$$

where $a, b \in \text{GF}(p)$ satisfy the non-singularity condition

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}.$$

⁶ We do not include [CFS26, LYW⁺26, BZG⁺26] in the comparison of the main body, since they report only the qubit and Toffoli counts, while depth is a key evaluation metric in our work. A partial comparison is provided in Appendix G.

Here, $p > 3$. The points on elliptic curve are the ordered pairs (x, y) whose coordinates are elements of $\text{GF}(p)$ and that satisfy the defining equation together with the special point $\mathcal{O}$. This set forms a group under the operation of addition on elliptic curves via the “chord-and-tangent” rule, where the point at infinity serves as the identity element

(ii) Montgomery Form. A Montgomery curve is defined by

$$M_{A,B} : \quad By^2 \equiv x^3 + Ax^2 + x \pmod{p},$$

where $A, B \in \text{GF}(p)$ with $B \neq 0$ and $A \neq \pm 2$, and p is an odd prime. The elliptic curve points are all $(x, y) \in \text{GF}(p)^2$ satisfying the defining equation, together with the point at infinity $\mathcal{O}$.

(iii) Twisted Edwards Form. A twisted Edwards curve is defined by

$$E_{a,d} : \quad ax^2 + y^2 \equiv 1 + dx^2y^2 \pmod{p},$$

where $a, d \in \text{GF}(p)$ with $a \neq 0$, $d \neq 0$, p is an odd prime, a is a square in $\text{GF}(p)$ while d is not, and $a \neq d$. The set of points on the elliptic curve consists of all $(x, y) \in \text{GF}(p)^2$ satisfying the equation. The identity element in this representation is $(0, 1)$. An Edwards curve is a twisted Edwards curve with $a = 1$. Group law for prime elliptic curves is discussed in Appendix A. An example of toy prime elliptic curve is given in Example 4. Some standard examples of prime elliptic curves are given in Examples 5, 6, and 7 for completeness. Key establishment using prime elliptic curves is described in Appendix B

2.3 Elliptic Curve Cryptography vs. Shor’s Algorithm

With the advent of quantum computing, the hardness assumption of the ECDLP is fundamentally challenged. Shor’s algorithm [Sho94] can solve this problem efficiently on a quantum computer, thereby threatening the security of ECC. The algorithm takes advantage of quantum parallelism and the Quantum Fourier Transform (QFT), which together enable exponential speedup over classical methods. Its quantum procedure can be summarized as follows:

Quantum Register Preparation: Initialize two quantum registers, k and ℓ , each consisting of $n+1$ qubits, along with a third register for group element computations. All registers start in the $|0\rangle$ state, where the third register is initialized to $|O\rangle$, the elliptic curve identity: $|0\rangle^{\otimes n+1} \otimes |0\rangle^{\otimes n+1} \otimes |O\rangle$. Next, apply Hadamard gates to every qubit in the first two registers, creating a uniform superposition over all possible values of k and ℓ :

$$\frac{1}{2^{n+1}} \sum_{k=0}^{2^{n+1}-1} \sum_{\ell=0}^{2^{n+1}-1} |k\rangle |\ell\rangle |O\rangle.$$

Oracle Mapping (Point Addition): For each pair (k, ℓ) , compute the linear combination of the two base points in the third register, which yields the following:

$$\frac{1}{2^{n+1}} \sum_{k, \ell} |k\rangle |\ell\rangle |O\rangle \rightarrow \frac{1}{2^{n+1}} \sum_{k, \ell} |k\rangle |\ell\rangle |[k]P + [\ell]Q\rangle.$$

Quantum Fourier Transform (QFT): A QFT is applied on both the k and ℓ registers, mapping the computational basis to the frequency domain. This transformation is essential for period finding:

$$|x\rangle \rightarrow \frac{1}{2^{n+1}} \sum_{y=0}^{2^{n+1}-1} e^{2\pi i xy / 2^{n+1}} |y\rangle.$$

Measurement and Post-processing: Measure the first two registers to obtain output values, which are used in classical post-processing to recover the discrete logarithm m . The third register is unmeasured and is discarded at the end.

Figure 1 shows a schematic of this quantum circuit, including initialization, oracle application, QFT, and measurement stages. Shor’s algorithm can also be implemented with a single control qubit using the semi-classical Fourier transform [GN96], and this approach is adopted in our work (see Appendix D for details). It is important to note that Ekerå [Eke19] showed (heuristically) that using the semi-classical Fourier transform reduces the single-run success probability of Shor’s algorithm for the discrete logarithm problem to about 60-82%, and proposed two strategies to recover a high success rate (above 99%): slightly increasing the quantum computation and performing a single limited search in the classical post-processing, or performing two limited searches in the classical post-process. In this work, the latter setting is assumed throughout our analysis without increasing the quantum cost (for details, see [Eke19]).

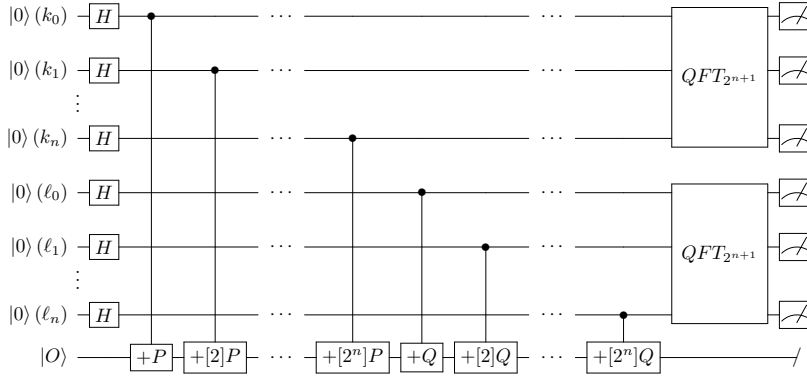


Figure 1: Circuit of Shor’s algorithm for solving ECDLP.

2.4 Quantum Gates

Figure 2 illustrates the quantum gates employed to construct quantum circuits in this work, where the symbols ‘ $\neg$ ’, ‘ $\oplus$ ’ and ‘ $\wedge$ ’ denote the NOT, XOR, and AND operations respectively. The Hadamard (H) gate (Figure 2(a)) creates superposition by assigning equal probability amplitudes to the $|0\rangle$ and $|1\rangle$ states. The X gate (Figure 2(b)) flips a qubit state between $|0\rangle$ and $|1\rangle$, corresponding to the classical NOT operation ($\neg$). The CNOT gate (Figure 2(c)) flips the target qubit (b) conditioned on the control qubit (a) being in the $|1\rangle$ state, realizing the classical XOR operation ($\oplus$). The Toffoli gate (Figure 2(d)) flips the target qubit (c) only when both control qubits (a and b) are in the $|1\rangle$ state, corresponding to the classical AND operation ($\wedge$). Due to its high quantum cost, various optimized constructions of the Toffoli gate have been proposed [AMM⁺13, Sel13]. In this work, we adopt a Toffoli decomposition which uses 7 T gates with a T -depth of 4 [AMM⁺13].

Replacing Toffoli gates with AND gates [Gid18, JNRV19] can further reduce the quantum cost. To the best of our knowledge, the most efficient AND gate constructions were proposed by Jaques et al. [JNRV19], and the corresponding circuits are illustrated in Figures 2(e) and 2(f). The AND gate is decomposed into 11 Clifford gates and 4 T gates, achieving a T -depth of 1 and a full depth of 8, while requiring one ancilla qubit. Its reverse operation, the $\text{AND}^\dagger$ gate, consists of 7 Clifford gates and a single measurement gate, providing a notable advantage in reverse operations. Note that the target qubit of the AND gate must be initialized in a clean state $|0\rangle$, in contrast to the Toffoli gate, whose target qubit does not require such initialization.

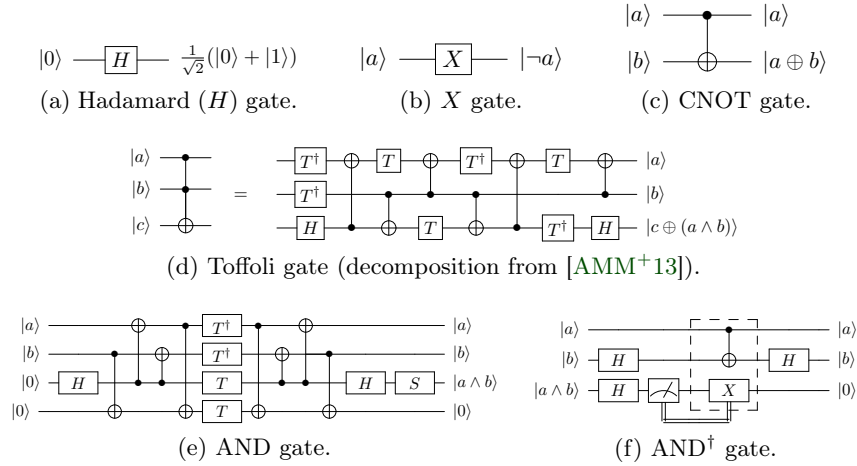


Figure 2: Quantum gates.

3 Quantum Circuits for Modular Arithmetic

In this section, we present quantum modular arithmetic circuits for Shor’s attack on elliptic curve cryptography. These circuits serve as building blocks for the point addition circuits in Section 4. Throughout our implementations, we use the AND gates presented in [JNRV19] instead of Toffoli gates (see Figure 2).

3.1 Modular Addition

We employ the in-place and out-of-place Draper adders [DKRS04], which are effective in reducing depth. Modular addition is implemented using a combination of these adders, as illustrated in Figure 3. After an addition, the modulus p is subtracted, followed by a conditional addition of p to keep the result in $[0, p - 1]$, as in [HJN⁺20].

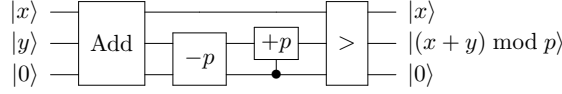


Figure 3: Modular addition (used in [HJN⁺20] and this paper).

Controlled Addition. Controlled addition is required in the point addition circuit to realize an in-place structure (see Figure 8). As shown in Figures 4(a) and 4(b), there are two approaches to controlled addition: reconstructing the adder and copying the operand. Both [RNSL17] and [HJN⁺20] adopt the first approach, which promotes the CNOT gates except those in the carry-propagation to Toffoli gates. In contrast, we adopt the latter approach, in which the operand is copied to clean ancilla qubits conditioned on the control qubit q , i.e., only when $q = 1$, and an uncontrolled addition is then performed without additional Toffoli gates. We make this choice because sufficient idle ancilla qubits are available in our implementation, as will become clear in later sections.

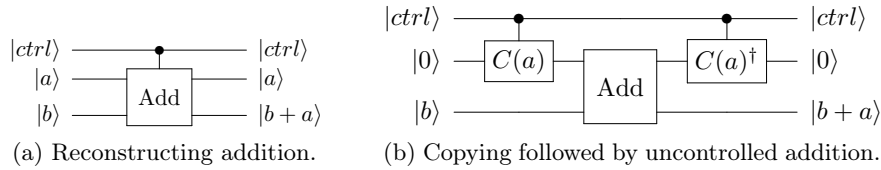


Figure 4: Comparison of two controlled addition methods.

3.2 Modular Multiplication

Both [RNSL17] and [HJN⁺20] adopt the Montgomery algorithm for quantum modular multiplication, but realize it with different circuit structures. The quantum circuit diagrams from [RNSL17] and [HJN⁺20] are shown in Appendix E (Figures 13(a) and 13(b), respectively). In [RNSL17], a bit-level Montgomery multiplication circuit is used. For each round $i = 0, \dots, n - 1$, the circuit performs

$$c(x) \leftarrow c(x) + a_i(x)b(x), \quad c(x) \leftarrow c(x) + c_0(x)p(x),$$

corresponding to the *general multiplication* and the *Montgomery reduction*, respectively. Here, $a_i(x)$ and $c_0(x)$ each act as a control qubit.

In contrast, [HJN⁺20] implements a word-level Montgomery multiplication with word size w , as given in Algorithm 1(a). Let $s = n/w$, and let $a_i(x)$ denote the w -qubit word for $i = 0, \dots, s - 1$. Then the general multiplication

$$c(x) \leftarrow c(x) + a_i(x)b(x)$$

corresponds to a multiplication of size $w \times n$, and is realized by w controlled additions of n -qubit operands. For the Montgomery reduction, letting $c_w(x)$ denote the least significant w qubits of $c(x)$, the update is written as

$$M(x) \leftarrow c_w(x)p'_w(x) \bmod x^w, \quad c(x) \leftarrow c(x) + M(x)p(x).$$

Here, $c_w(x)$ is used as the index of the quantum lookup table, which loads the corresponding multiple $M(x)p(x)$. Hence, the w controlled additions required for the Montgomery reduction are replaced by a single uncontrolled addition assisted by the quantum lookup table. Note that the cost of constructing the quantum lookup table should be taken into account. We discuss this trade-off in detail in Appendix F.

QCSA-based Montgomery Multiplication. In this work, we present a word-level (of size w) Montgomery multiplication using the Quantum Carry-Save Adder (QCSA) [KLJ⁺25], which is similar to the approach in [HJN⁺20] but differs in key aspects. Depending on the word size w , Montgomery multiplication is performed between each input $a_i(x)$ ($0 \leq i \leq s - 1$, $s = n/w$) and $b(x)$. Here, the sizes of $a_i(x)$ and $b(x)$ are w and n , respectively.

Figure 5 illustrates a single loop (iterated s times) of our QCSA-based Montgomery multiplication (C denotes a copy operation). The left and right sides of the dashed line correspond to the general multiplication and the Montgomery reduction, respectively. In our implementation, we employ the QCSA, a quantum adder optimized for multi-operand addition (e.g., $a + b + c$ corresponds to a three-operand addition).

Phase 1: Input Setup. As a preparatory step, we prepare w registers $QCSA_{in}$, each consisting of n qubits, and store $b(x)$ in each register conditioned on the

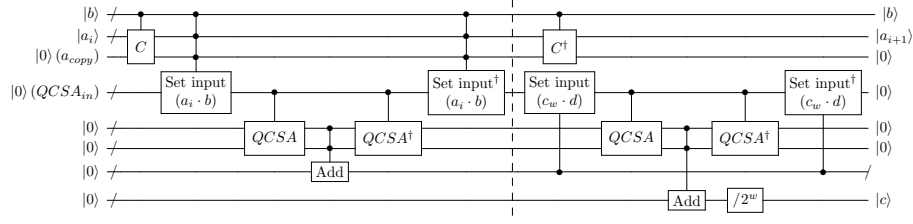


Figure 5: Montgomery multiplication circuits.

corresponding control qubit $a_i(x)$ using AND gates⁷. Each of the w registers is set to $b(x)$ (if $a_i = |1\rangle$) or to $|0\rangle$ (if $a_i = |0\rangle$). To improve the parallelization of AND gates in this step, we copy (C) the input $a_i(x)$ into ancilla qubits (a_{copy}) using CNOT gates. Assuming the input is not copied, only w AND gates can be executed in parallel.

Phase 2: Multi-Operand Addition Using QCSA. Using the prepared w registers, we perform multi-operand addition with the QCSA (QCSA in Figure 5), which computes the result of $a_i(x)b(x)$. The QCSA is performed until only two operands remain. For the final two-operand addition (Add⁸ in Figure 5), we adopt Draper’s out-of-place adder [DKRS04]. In contrast to [RNSL17,HJN⁺20], where sequential quantum additions are designed for general multiplication $a(x)b(x)$, our implementation constructs a parallelized addition using the QCSA.

Phase 3: Reuse of Ancilla Qubits. We initialize the ancilla qubits used in the QCSA to the clean state by applying its reverse operation ($QCSA^\dagger$). The initialized ancilla qubits are then reused in subsequent QCSA. In the same context, we reverse the input setup operation using AND[†] gates (Set input[†]).

Phase 4: Alternative Formulation of Montgomery Reduction. After the general multiplication $a_i(x)b(x)$, the Montgomery reduction is performed, corresponding to the right side of the dashed line in Figure 5. To enable the use of the QCSA for the Montgomery reduction, we reformulate the word-level Montgomery multiplication in Algorithm 1(a) into the alternative form shown in Algorithm 1(b). In Algorithm 1(a), which describes the general Montgomery multiplication, the reduction step is given by

$$M(x) \leftarrow c_w(x)p'_w(x) \bmod x^w, \quad c(x) \leftarrow c(x) + M(x)p(x).$$

Here, $M(x)$ is an intermediate value determined by the current word $c_w(x)$. Substituting the expression for $M(x)$ into the second update gives

$$c(x) \leftarrow c(x) + c_w(x)((p'_w(x) \bmod x^w)p(x)).$$

⁷ The control qubits are $a_i(x)$ and $b(x)$, and the w registers serve as the target qubits.

⁸ Note that the connected dots represent operands for out-of-place addition, not controlled addition.

Since the factor depending on the modulus $p(x)$ and $p'(x)$ can be precomputed classically, we define the constant $d(x) := (p'_w(x) \bmod x^w)p(x)$. Then the reduction step can be rewritten as

$$c(x) \leftarrow c(x) + c_w(x)d(x),$$

which corresponds to line 5 of Algorithm 1(b). Similarly to the general multiplication $a_i(x)b(x)$, the precomputed factor $d(x)$ is loaded into the w registers according to the input $c_w(x)$, thus the product $c_w(x)d(x)$ can be computed using the QCSA followed by Draper's out-of-place adder. Finally, the reverse operations of the QCSA and the input setup are performed, followed by a logical shift ($/2^w$) on the result $c(x)$.

Algorithm 1: Word-level Montgomery multiplication (of size $n = sw$) and its equivalent formulation (by us).

(a) Word-level Montgomery.	(b) Equivalent formulation.
Input: $a(x), b(x), p(x), p'_w(x)$	Input: $a(x), b(x), p(x), p'_w(x)$
Out: $c(x) = a(x)b(x)r^{-1}(x) \bmod p(x)$	Out: $c(x) = a(x)b(x)r^{-1}(x) \bmod p(x)$
1: $c(x) \leftarrow 0$	1: $c(x) \leftarrow 0$
2: for $i = 0$ to $s - 1$ do	2: $d(x) \leftarrow (p'_w(x) \bmod x^w)p(x)$
3: $c(x) \leftarrow c(x) + a_i(x)b(x)$	3: for $i = 0$ to $s - 1$ do
4: $M(x) \leftarrow c_w(x)p'_w(x) \bmod x^w$	4: $c(x) \leftarrow c(x) + a_i(x)b(x)$
5: $c(x) \leftarrow c(x) + M(x)p(x)$	5: $c(x) \leftarrow c(x) + c_w(x)d(x)$
6: $c(x) \leftarrow c(x)/x^w$	6: $c(x) \leftarrow c(x)/x^w$
7: end for	7: end for
8: return $c(x)$	8: return $c(x)$

Qubit-Optimized Version. We also implement a qubit-optimized version of the QCSA-based multiplication, whereas the version presented so far is the depth-optimized one. There are two modifications in the qubit-optimized version. First, the qubit-optimized version uses a small word size, $w = 4$, to reduce the number of ancilla qubits required for the QCSA. For comparison, the depth-optimized version uses $(n, w) = (192, 64), (224, 56), (256, 64), (384, 96)$, and $(512, 128)$. Second, we modify the two-operand addition step, which is implemented with Draper's out-of-place adder (Add in Figure 5). In the depth-optimized version, output qubits are newly allocated for each two-operand addition. In contrast, the qubit-optimized version uncomputes each two-operand addition and reuses the output qubits.

Performance. Table 1 shows the quantum resources required for modular multiplication in [HJN⁺20] and in this work. We exclude [RNSL17] from the comparison, as [HJN⁺20] is a follow-up work that reports improved performance. Additionally, because [HJN⁺20] contains an issue in its lower-bound estimation, all comparisons in this paper are based on our reimplementations of [HJN⁺20]

(see Section 5 for details). The work by Gouzien et al. [GRLR⁺23] is excluded from the comparison because it is affected by the same issue as [HJN⁺20], since their implementation is based on that of [HJN⁺20].

We achieve low circuit depth and gate count, but this comes with a higher qubit count. In this trade-off, our multiplication provides performance improvements of approximately 62%–85% and 60%–93% in terms of the qubit count – T -depth (TD - M) and qubit count – full depth products (FD - M), respectively, compared to the depth-optimized version in [HJN⁺20], as shown in Table 1.

3.3 Modular Inversion

This section presents our quantum inversion circuit, a building block of the division procedure (Section 3.4). For inversion, we adopt the binary Extended Euclidean Algorithm (EEA) and Kaliski’s algorithm [Kal02]. This approach was adopted in [RNSL17], and further improved in [HJN⁺20].

Full-Circuit Execution without Conditionals. We adopt the swap-based Kaliski’s algorithm specified in Algorithm 2, a refined version of the binary EEA presented in [HJN⁺20, Figure 7]. At each round, the variables (u, r, v, s) are updated according to the values of u and v . This process continues until $v = 0$, at which point $u = 1$ and $s = p$, and r contains a pseudo-inverse of x . Formally (or classically), each round is executed only if the condition $v_i \neq 0$ holds, and the state v_i is updated according to Kaliski’s algorithm. A key difference in our implementation is the removal of the check step for whether $v = 0$, resulting in unconditional execution. In [HJN⁺20], the round operation performs conditional checks over the range $n \leq i < 2n$ to determine whether the state needs to be updated (for details, refer to Section 5). In contrast, we execute all $2n$ rounds without any conditional checks (at low cost).

Pseudo-Inverse Correction. In [HJN⁺20], the pseudo-inverse is given by $x^{-1} \cdot 2^{-n+k} \pmod{p}$, where k denotes the number of activated round updates (i.e., the rounds that are actually executed). To recover the correct Montgomery inverse, $x^{-1} \cdot 2^n \pmod{p}$, the $2n-k$ doublings are performed using the counter that stores the number of activated rounds (k) in the inversion. In our implementation, since the round operations are unconditional, the resulting value is already given as $x^{-1} \cdot 2^n (= x^{-1} \cdot 2^{-n} \cdot 2^{2n})$, which is in Montgomery inverse form. However, this requires a larger register to represent the value on $2n$ qubits.

Low-Depth Round Operation. The round operation is structurally similar to that in [HJN⁺20]; however, we improve the circuit depth by enhancing parallelism. Figure 6 illustrates a round operation of our inversion quantum circuit. We copy (C) the control qubit a (which checks the state of u and v) into a sufficient number of ancilla qubits (a_{copy}) and parallelize all controlled operations that use the control qubit a . Similarly, the operands u and r are also copied to ancilla qubits (i.e., u_{copy} and r_{copy}), and the uncontrolled addition and subtraction with

Table 1: Comparison of the quantum resources required for multiplication.

n	Source	#CNOT	#Measure	#T	#Qubit (M)	T -depth (TD)	Full depth (FD)	TD - M cost*	FD - M cost*
192	H ⁺ [HJN ⁺ 20]	Q opt. 6897744	24384	1819200	1002	740736	5003072	$2^{29.47}$	$2^{32.23}$
		D opt. 26854848	270768	5511936	2500	38928	713616	$2^{26.54}$	$2^{30.74}$
	This paper	Q opt. 7670988	644360	2879264	4249	8102	129140	$2^{25.04}$ (65%)	$2^{29.03}$ (69%)
		D opt. 2259229	151529	613796	43210	390	3736	$2^{24.01}$ (83%)	$2^{27.27}$ (91%)
224	H ⁺ [HJN ⁺ 20]	Q opt. 6837768	28448	2448544	1162	1003968	5234260	$2^{30.12}$	$2^{32.51}$
		D opt. 33971728	363272	7466032	2901	45920	832496	$2^{26.99}$	$2^{31.17}$
	This paper	Q opt. 10341274	875396	3911056	4943	9670	135176	$2^{25.51}$ (64%)	$2^{29.32}$ (72%)
		D opt. 2886116	192736	782008	42457	470	4425	$2^{24.25}$ (85%)	$2^{27.49}$ (92%)
256	H ⁺ [HJN ⁺ 20]	Q opt. 8259008	32512	3171072	1322	1307136	6481948	$2^{30.69}$	$2^{33.00}$
		D opt. 43661824	471616	9741312	3302	53056	1025728	$2^{27.39}$	$2^{31.66}$
	This paper	Q opt. 13500588	1145864	5116960	5642	11302	160274	$2^{25.93}$ (64%)	$2^{29.75}$ (73%)
		D opt. 3729175	253862	1028092	55267	511	4550	$2^{24.75}$ (84%)	$2^{27.91}$ (93%)
384	H ⁺ [HJN ⁺ 20]	Q opt. 24062112	48768	6993024	1962	2919168	17344576	$2^{32.42}$	$2^{34.99}$
		D opt. 103205568	1037472	21769152	4909	81312	1550592	$2^{28.58}$	$2^{32.83}$
	This paper	Q opt. 30513830	2575876	11496976	8424	18090	327228	$2^{27.18}$ (62%)	$2^{31.36}$ (63%)
		D opt. 8420203	561477	2264980	120435	666	7465	$2^{26.26}$ (81%)	$2^{29.74}$ (88%)
521	H ⁺ [HJN ⁺ 20]	Q opt. 50627468	67056	12752248	2384	5358846	36908138	$2^{33.58}$	$2^{36.36}$
		D opt. 198265188	1912944	40473312	6624	114794	2285052	$2^{29.51}$	$2^{33.82}$
	This paper	Q opt. 54415510	4581380	20441104	11209	25642	543940	$2^{28.10}$ (62%)	$2^{32.51}$ (60%)
		D opt. 14721502	974348	3922928	208654	794	9035	$2^{27.30}$ (78%)	$2^{30.81}$ (88%)

*: Improvement percentages are relative to the D opt. results in [HJN⁺20].

copying are implemented (see Figure 4(b)). Note that the ancilla qubits used for copying ($a_{copy}, u_{copy}, r_{copy}$) are borrowed from the idle qubits reserved for the subsequent multiplication following the inversion (a characteristic of division); thus, this does not increase the overall qubit count.

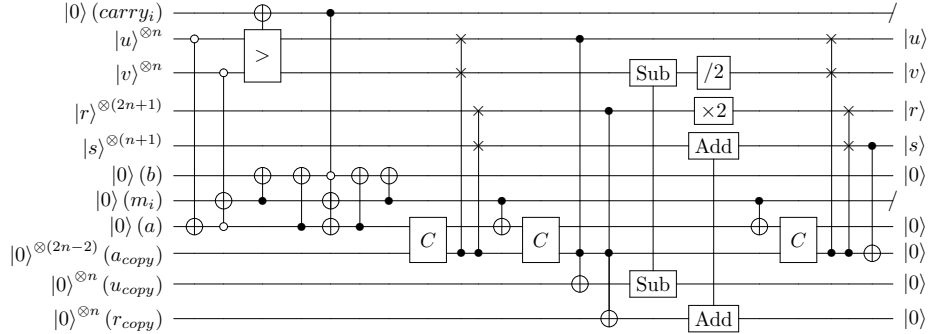


Figure 6: Round operation of inversion (one round).

Postponed Modular Reduction. For the subtraction ($v - u$, Line 10), addition ($r + s$, Line 11), halving ($v/2$, Line 13), and doubling ($2 \cdot r$, Line 14)

Algorithm 2: Kaliski’s algorithm (equivalent formulation by [HJN⁺20]).

Input: u, v, r, s Output: Pseudo-inverse r 1: while $v \neq 0$ do 2: $b_{\text{swap}} \leftarrow \text{false}$ 3: if u even and v odd, or u and v 4: both odd and $u > v$ then 5: swap u and v 6: swap r and s 7: $b_{\text{swap}} \leftarrow \text{true}$ 8: end if	9: if u and v both odd then 10: $v \leftarrow v - u$ 11: $s \leftarrow r + s$ 12: end if 13: $v \leftarrow v/2$ 14: $r \leftarrow 2 \cdot r$ 15: if b_{swap} then 16: swap u and v 17: swap r and s 18: end if 19: end while 20: return r
--	---

operations in Algorithm 2, we do not perform modular reduction (mod p) immediately after each operation. Instead, we postpone these modular reductions by accumulating the results into a $2n$ -bit inverse. Thanks to this strategy, the cost of modular reduction is excluded from each round and instead incurred only at the final stage (we employ the QCSA for this reduction). One might wonder about the possibility of further postponing the modular reductions and performing them together after division. However, this would increase the size of the multiplication (i.e., $y \cdot x^{-1}$) to $n \times 2n$, which increases the cost; therefore, we do not adopt this approach.

3.4 Modular Division (and Performance)

Figure 7 shows the quantum circuit for the division, $\lambda = y \cdot x^{-1}$. Note that λ is conditionally stored in the result register depending on the control qubit q , after which the reverse operations ($\dagger$) of the modular reduction, multiplication, and inversion are performed. Table 2 compares the quantum resources required for division. We compare our results with the depth-optimized division in [HJN⁺20]. In terms of the products of T -depth and qubit count (TD - M) and full depth and qubit count (FD - M), our division achieves improvements of 58%–60% and 35%–40%, respectively.

4 Quantum Circuits for Point Addition

In this section, we present quantum point addition circuits for elliptic curves. We introduce two types of circuits: in-place and out-of-place point additions, which are implemented in affine and projective coordinates, respectively.

4.1 In-Place Point Addition (in Affine Coordinates)

In Shor’s algorithm, the point addition in affine coordinates, $P_1(x_1, y_1) + P_2(x_2, y_2)$, is performed when the control qubit q is 1; otherwise, the input point $P_1(x_1, y_1)$ is

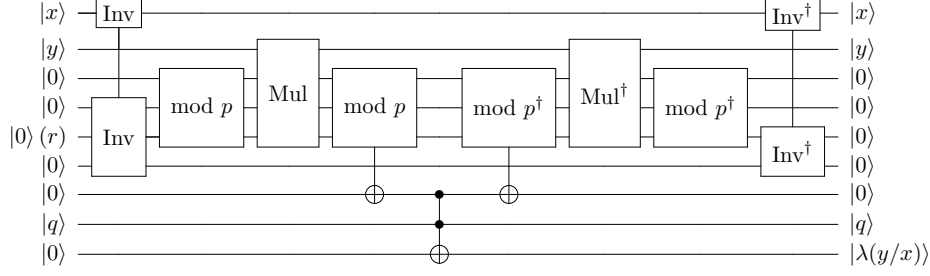


Figure 7: Modular division.

Table 2: Comparison of the quantum resources required for division.

n	Source	#CNOT	#Measure	#T	#Qubit (M)	T -depth (TD)	Full depth (FD)	TD - M cost*	FD - M cost*
192	H^+ [HJN+20]	Q opt. 29730087	24384	21042570	1656	7557624	23688176	$2^{33.55}$	$2^{35.20}$
		D opt. 123649134	2471643	47109801	3076	185613	1722357	$2^{29.09}$	$2^{32.31}$
	This paper	44527810	3155350	17222520	5593	40442	573020	$2^{27.76}$ (60%)	$2^{31.58}$ (40%)
224	H^+ [HJN+20]	Q opt. 38786015	28448	29275855	1932	10275928	27812635	$2^{34.21}$	$2^{35.65}$
		D opt. 166014859	3370916	64205236	3573	215657	2040367	$2^{29.53}$	$2^{32.77}$
	This paper	59197200	4162022	23366816	6511	48198	668590	$2^{28.23}$ (59%)	$2^{32.02}$ (40%)
256	H^+ [HJN+20]	Q opt. 48794519	32512	37278607	2188	13411000	35541507	$2^{34.78}$	$2^{36.18}$
		D opt. 216503029	4416866	84023622	4070	259797	2441168	$2^{29.98}$	$2^{33.21}$
	This paper	77098620	6728410	29829168	7434	60280	796034	$2^{28.74}$ (58%)	$2^{32.47}$ (40%)
384	H^+ [HJN+20]	Q opt. 117811575	48768	85695375	3285	30118968	68929575	$2^{36.53}$	$2^{37.73}$
		D opt. 493942005	9989802	189705726	6061	412749	3806139	$2^{31.23}$	$2^{34.43}$
	This paper	179584894	12793432	69515064	11112	90530	1315624	$2^{29.91}$ (60%)	$2^{33.77}$ (37%)
521	H^+ [HJN+20]	Q opt. 223115409	67056	157542194	4432	55396135	141840206	$2^{37.84}$	$2^{39.20}$
		D opt. 920615553	18518034	350963624	8187	598747	5583108	$2^{32.20}$	$2^{35.42}$
	This paper	356484140	26166202	138328000	14856	134126	2002592	$2^{30.90}$ (59%)	$2^{34.80}$ (35%)

Our division uses the qubit-optimized multiplication.

*: Improvement percentages are relative to the D opt. results in [HJN+20].

preserved. Algorithm 3 and Figure 8 present our in-place point addition circuit, adapted from [HJN+20] with two modifications.

Modifications from HJNRS (PQCrypto’20) Our first modification concerns the handling of the control qubit q . In the point addition circuit of Figure 8, three subtractions, two divisions, and one negation are conditionally performed depending on q . If these controlled operations use a single control qubit, the inner quantum gates are executed sequentially. To avoid this, we copy q into the ancilla registers used for multiplication and inversion, namely M_{anc} and I_{anc} , while those registers are idle (C in Figure 8).

Our second modification is to merge the final steps of [HJN+20], which consist of an uncontrolled subtraction followed by a controlled addition. The update

Algorithm 3: Quantum circuit of in-place point addition.

Quantum input: Accumulated point $P_1(x_1, y_1)$, point $P_2(x_2, y_2)$, and clean ancilla qubits

Output: Updated accumulated point $P_3(x_3, y_3)$ and clean ancilla qubits.

1: $x \leftarrow \text{Sub}(x_1, x_2) \triangleright x = x_1 - x_2$ 2: $y \leftarrow \text{Ctrl_Sub}(y_1, qy_2) \triangleright y = y_1 - qy_2$ 3: $\lambda \leftarrow \text{Ctrl_Div}^\oplus(q, x_1, y_1) \triangleright \lambda = q \cdot \frac{y}{x}$ 4: $y \leftarrow \text{Mul}^\oplus(x_1, \lambda, y_1) \triangleright y \leftarrow y \oplus (x\lambda)$ 5: $x \leftarrow \text{Add}(x_1, 3x_2) \triangleright x \leftarrow x + 3x_2$ 6: $x \leftarrow \text{Mul}^-(\lambda, \lambda^{\text{copy}}, x_1) \triangleright x \leftarrow x - \lambda^2$	7: $y \leftarrow \text{Mul}^+(x_1, \lambda, y_1) \triangleright y \leftarrow y + x\lambda$ 8: $\lambda \leftarrow \text{Ctrl_Div}^\oplus(q, x_1, y_1, \lambda) \triangleright \lambda \leftarrow 0$ 9: $x \leftarrow \text{Ctrl_Sub}(x_1, qx_2 + \bar{q}2x_2) \triangleright x \leftarrow -x_3$ 10: $y \leftarrow \text{Ctrl_Sub}(y_1, qy_2) \triangleright y \leftarrow y_3$ 11: $x \leftarrow \text{Ctrl_Neg}(q, x_1) \triangleright x \leftarrow x_3$
--	--

in [HJN⁺20, Figure 9] first applies $x_1 \leftarrow x_1 - 2x_2$, followed by

$$x_1 \leftarrow \begin{cases} x_1 + x_2, & \text{if } q = 1, \\ x_1, & \text{if } q = 0, \end{cases} \implies x_1 \leftarrow \begin{cases} x_1 - x_2, & \text{if } q = 1, \\ x_1 - 2x_2, & \text{if } q = 0. \end{cases}$$

In our implementation, these two steps are combined into a single controlled subtraction. More precisely, using the controlled adder in Figure 4(b), we prepare an ancilla register storing

$$(2 - q)x_2 = \begin{cases} 2x_2, & \text{if } q = 0, \\ x_2, & \text{if } q = 1, \end{cases} \quad \text{and then perform} \quad x_1 \leftarrow x_1 - (2 - q)x_2,$$

which corresponds to Step 9 in Algorithm 3.

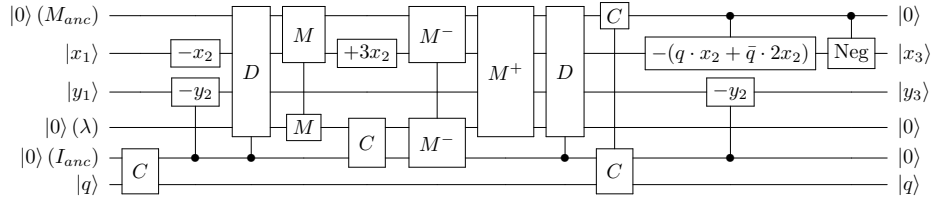


Figure 8: In-place point addition (Algorithm 3, modified from [HJN⁺20, Figure 9] by us).

4.2 Out-of-Place Point Addition (in Projective Coordinates)

This section describes our out-of-place point addition in projective coordinates. More precisely, we use Jacobian-affine mixed coordinates, where the accumulated point P_1 is $(X_1 : Y_1 : Z_1)$ and the point P_2 is $(x_2, y_2, 1)$. In Shor's circuit, P_2 is a classically known point, so it can be given directly in affine form with $Z_2 = 1$. For $P_3 = P_1 + P_2 = (X_3 : Y_3 : Z_3)$, we define

$$U_2 = x_2 Z_1^2, \quad S_2 = y_2 Z_1^3, \quad H = U_2 - X_1, \quad R = S_2 - Y_1.$$

The output coordinates are given by

$$X_3 = R^2 - H^3 - 2X_1H^2, \quad Y_3 = R(X_1H^2 - X_3) - Y_1H^3, \quad Z_3 = Z_1H.$$

This removes the terms involving Z_2 that appear in general Jacobian-Jacobian point addition. To the best of our knowledge, this method uses the fewest multiplications among quantum implementations in projective coordinates, requiring 11 multiplications, whereas the previous minimum was 12 [CFS26]. Note that the 12 multiplications refer to the algorithm level, whereas 34 multiplications are used in their implementation (see [CFS26, Section 2.2]).

Algorithm 4 and Figure 9 implement our point addition in projective coordinates in a straightforward manner and use our QCSA-based multiplication (Section 3.2, the depth-optimized version). The obvious advantage of projective coordinates is that point addition does not require inversion. Despite this, most previous works [RNSL17, HJN⁺20, JSB⁺25] have considered affine coordinates. This is mainly because the non-unique representation of projective coordinates produces garbage qubits in every point addition (resulting in an out-of-place design). We discuss this issue and demonstrate how it can be addressed.

Algorithm 4: Quantum circuit of out-of-place point addition.

Quantum input: Accumulated point $P_1(X_1, Y_1, Z_1)$, point $P_2(x_2, y_2)$ from lookup, and clean ancilla qubits.

Output: Updated accumulated point $P_3(X_3, Y_3, Z_3)$, clean ancilla qubits, and $P_1(X_1, Y_1, Z_1)$ as garbage qubits.

1: $Z_1^2 \leftarrow \text{Mul}(Z_1, Z_1^{\text{copy}})$	11: $2V \leftarrow \text{Add}(V, V^{\text{copy}})$
2: $U_2 \leftarrow \text{Mul}(x_2, Z_1^2)$	12: $T_0 \leftarrow \text{Sub}(R^2, H^3) \quad \triangleright T_0 = R^2 - H^3$
3: $Z_1^3 \leftarrow \text{Mul}(Z_1^2, Z_1)$	13: $X_3 \leftarrow \text{Sub}(T_0, 2V) \quad \triangleright X_3 = T_0 - 2V$
4: $H \leftarrow \text{Sub}(U_2, X_1) \quad \triangleright H = U_2 - X_1$	14: $T_1 \leftarrow \text{Sub}(V, X_3) \quad \triangleright T_1 = V - X_3$
5: $H^2 \leftarrow \text{Mul}(H, H^{\text{copy}})$	15: $T_2 \leftarrow \text{Mul}(R, T_1)$
6: $H^3 \leftarrow \text{Mul}(H^2, H)$	16: $T_3 \leftarrow \text{Mul}(Y_1, H^3)$
7: $S_2 \leftarrow \text{Mul}(y_2, Z_1^3)$	17: $Y_3 \leftarrow \text{Sub}(T_2, T_3) \quad \triangleright Y_3 = T_2 - T_3$
8: $R \leftarrow \text{Sub}(S_2, Y_1) \quad \triangleright R = S_2 - Y_1$	18: $Z_3 \leftarrow \text{Mul}(Z_1, H)$
9: $R^2 \leftarrow \text{Mul}(R, R^{\text{copy}})$	19: return (X_3, Y_3, Z_3)
10: $V \leftarrow \text{Mul}(X_1, H^2)$	

Limitations and Their Mitigation. A critical reason projective coordinates are not preferred for quantum point addition is the presence of garbage qubits. In projective coordinates, although $(P+Q)-Q \equiv P$ mathematically, the recovered point is bitwise different from the original P . Therefore, the input P cannot be uncomputed to zero from $(P+Q)$ and the known point Q , and remains as garbage after the point addition. This is particularly problematic in Shor’s algorithm, where garbage qubits are generated in every point addition. Despite this, we demonstrate that its practical impact is mitigated by the following two factors.

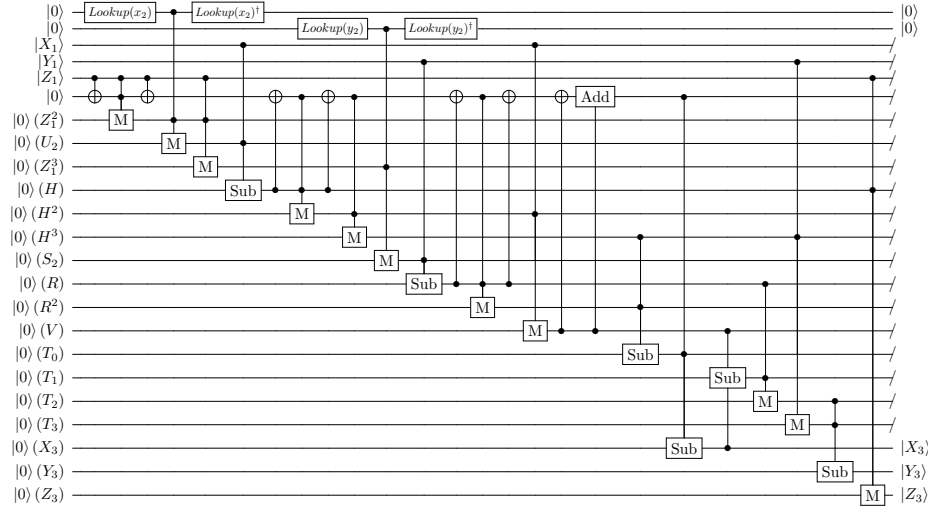


Figure 9: Out-of-place point addition (Algorithm 4, ours)

Mitigating Factor 1. In Shor’s algorithm, a total of $2n + 2$ conditional point additions are required, each controlled by q (see Figure 1). However, the windowing technique [GE21] reduces the total number of point additions in Shor’s algorithm. It groups the control qubits for point additions into w -qubit registers, where w denotes the window size. These registers serve as addresses for looking up the precomputed points $T, T + [1]P_2, T + [2]P_2, \dots, T + [2^w - 1]P_2$ in superposition, where a fixed point T is used to avoid the point at infinity. As a result, the windowed approach requires only $2 \cdot \lceil \frac{n+1}{w} \rceil$ point additions, thereby saving $w - 1$ additions per window⁹. Let G denote the number of garbage qubits generated by one point addition. Then, windowing reduces the total number of garbage qubits from $(2n + 2)G$ to $2 \lceil \frac{n+1}{w} \rceil G$.

Mitigating Factor 2. We introduce a zigzag architecture for Shor’s algorithm inspired by Grassl et al. [GLRS16]. This architecture was originally developed for AES quantum circuits, where the outputs of AES rounds are initialized and then reused in subsequent rounds. Figure 10 illustrates the zigzag architecture for Shor’s algorithm with 10 point additions. This initializes the previous outputs using the reverse operations ($PA^\dagger$ s in Figure 10) and only four output registers are allocated for 10 point additions. Conceptually, the point additions (PAs in Figure 10) are arranged in a zigzag form as $10 = 4 + 3 + 2 + 1$. In general, for $2 \lceil \frac{n+1}{w} \rceil$ point additions, the smallest integer m satisfying

$$\frac{m(m+1)}{2} \geq 2 \left\lceil \frac{n+1}{w} \right\rceil$$

⁹ For a window size w , each lookup table requires $(2^w - 4)$ Toffoli gates. Hence, the optimal window size must be chosen to minimize Shor’s cost (see Section 6.1).

determines the number of required output registers. Hence, the total number of garbage qubits is further reduced from $2 \lceil \frac{n+1}{w} \rceil G$ to mG . Let us assume the window size $w = 11$ for $n = 192$. The total number of point additions is $2 \lceil 193/11 \rceil = 36$, and the smallest m satisfying $m(m+1)/2 \geq 36$ is $m = 8$. Therefore, although 36 point additions are still performed, garbage qubits are required for only 8 of them.

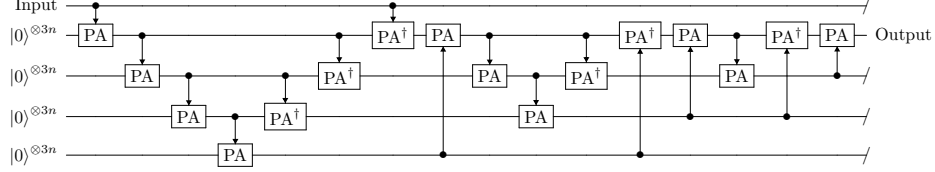


Figure 10: Zigzag architecture for Shor's algorithm with 10 point additions.

Return to Unique Representation. In Shor's algorithm, the group operation must define a well-formed function $f(k, \ell) = [k]P + [\ell]Q$, so that each output $f(k, \ell)$ corresponds to a single basis state. Thus, we have to return to affine coordinates to obtain a unique representation. This is done by computing the inverse Z^{-1} and converting (X, Y, Z) into (XZ^{-1}, YZ^{-1}) . It is important to note that this step is performed only once at the end of Shor's algorithm. Further, Z_3 is obtained early in our point addition ($U_2 = x_2 \cdot Z_1^2$, $H = U_2 - X_1$, $Z_3 = Z_1 \cdot H$), so Z_3^{-1} can be computed in parallel during the last point addition.

4.3 Performance

Table 3 shows the quantum resources required for point addition. Compared to [HJN⁺20], both our in-place and out-of-place point additions achieve lower total gate count and depth, at the cost of more qubits. For the in-place point addition, the $TD-M$ and $FD-M$ are improved by 58%–61% and 43%–52%, respectively. For the out-of-place point addition, we use even more qubits, but achieve much lower gate count and depth. The out-of-place point addition provides the best performance in terms of the qubit count – depth products. Specifically, for $TD-M$ and $FD-M$, it achieves improvements of 81%–82% and 82%–87%, respectively.

4.4 Extension to Ed25519

Our out-of-place point addition strategy can also be applied to Ed25519 [BDL⁺12], a widely used signature scheme based on a twisted Edwards curve over $\mathbb{F}_p$ with $p = 2^{255} - 19$. In this section, we extend the out-of-place point addition from Section 4.2 to the curve underlying Ed25519. The extension uses extended Edwards coordinates $(X : Y : Z : T)$, which satisfy

$$x = X/Z, \quad y = Y/Z, \quad T = XY/Z.$$

Table 3: Comparison of the quantum resources required for point addition.

n	Source	Total gates	#Qubit (M)	T -depth (TD)	Full depth (FD)	TD - M^*	FD - M^*
192	$H^+[HJN+20]$	In-place (Q opt)	$2^{26.93}$	1665	17376420	$2^{34.76}$	$2^{36.60}$
		(D opt)	$2^{28.73}$	3085	489064	$2^{30.50}$	$2^{34.01}$
	This paper	In-place	$2^{27.29}$ (63%)	5601	105949	$2^{29.15}$ (61%)	$2^{33.01}$ (50%)
		Out-of-place	$2^{25.00}$ (92%)	59635	4847	$2^{28.11}$ (81%)	$2^{31.25}$ (85%)
224	$H^+[HJN+20]$	In-place (Q opt)	$2^{27.29}$	1941	23609252	$2^{35.42}$	$2^{37.02}$
		(D opt)	$2^{29.14}$	3582	570162	$2^{30.93}$	$2^{34.46}$
	This paper	In-place	$2^{27.71}$ (63%)	6519	126198	$2^{29.62}$ (60%)	$2^{33.41}$ (52%)
		Out-of-place	$2^{25.35}$ (93%)	67614	5727	$2^{28.53}$ (81%)	$2^{31.67}$ (86%)
256	$H^+[HJN+20]$	In-place (Q opt)	$2^{27.62}$	2197	30795428	$2^{35.98}$	$2^{37.54}$
		(D opt)	$2^{29.53}$	4079	679884	$2^{31.37}$	$2^{34.92}$
	This paper	In-place	$2^{28.10}$ (63%)	7442	155291	$2^{30.11}$ (58%)	$2^{33.85}$ (52%)
		Out-of-place	$2^{25.72}$ (93%)	82970	6180	$2^{28.93}$ (82%)	$2^{32.00}$ (87%)
384	$H^+[HJN+20]$	In-place (Q opt)	$2^{28.90}$	3294	69073572	$2^{37.73}$	$2^{39.19}$
		(D opt)	$2^{30.72}$	6070	1070624	$2^{32.60}$	$2^{36.12}$
	This paper	In-place	$2^{29.30}$ (63%)	11120	236188	$2^{31.30}$ (60%)	$2^{35.23}$ (46%)
		Out-of-place	$2^{26.89}$ (93%)	155992	7806	$2^{30.18}$ (81%)	$2^{33.61}$ (82%)
521	$H^+[HJN+20]$	In-place (Q opt)	$2^{29.83}$	4441	126974888	$2^{39.04}$	$2^{40.68}$
		(D opt)	$2^{31.62}$	8196	1543168	$2^{33.56}$	$2^{37.11}$
	This paper	In-place	$2^{30.26}$ (61%)	14864	346135	$2^{32.27}$ (59%)	$2^{36.29}$ (43%)
		Out-of-place	$2^{27.69}$ (93%)	248019	9333	$2^{31.11}$ (82%)	$2^{34.56}$ (83%)

※: Improvement percentages are relative to the in-place (D opt.) results in [HJN+20].

Let the accumulated point be $P_1 = (X_1 : Y_1 : Z_1 : T_1)$ and the classically known point be $P_2 = (x_2, y_2)$. Then, for $P_3 = P_1 + P_2 = (X_3 : Y_3 : Z_3 : T_3)$, where d denotes the curve parameter of Ed25519, the mixed point addition formulas are given by

$$A = (Y_1 - X_1)(y_2 - x_2), \quad B = (Y_1 + X_1)(y_2 + x_2), \quad C = 2dT_1x_2y_2, \quad D = 2Z_1,$$

$$E = B - A, \quad F = D - C, \quad G = D + C, \quad H = B + A.$$

The output coordinates are then

$$X_3 = EF, \quad Y_3 = GH, \quad Z_3 = FG, \quad T_3 = EH.$$

Figure 11 illustrates the quantum circuit for point addition on the Ed25519 curve. A notable difference is that point addition for Ed25519 can be implemented using only 7 multiplications, whereas our point addition for prime Weierstrass curves requires 11. For windowing, we load the values $(y_2 - x_2)$, $(y_2 + x_2)$, and $2dx_2y_2$ from the quantum lookup table. The required quantum resources for point addition on Ed25519 are reported in Table 4.

Note that affine Edwards coordinates require more operations than affine Weierstrass coordinates for in-place point addition. Thus, it is preferable to transform the fixed points P and Q offline from the Edwards curve to a Weierstrass curve via an isomorphic mapping [MS16], so that the in-place point addition of Section 4.1 can be applied directly. As a result, the in-place implemen-

tation for Ed25519 requires resources comparable to those of ECC-256, with a slight increase due to the difference in modulus.

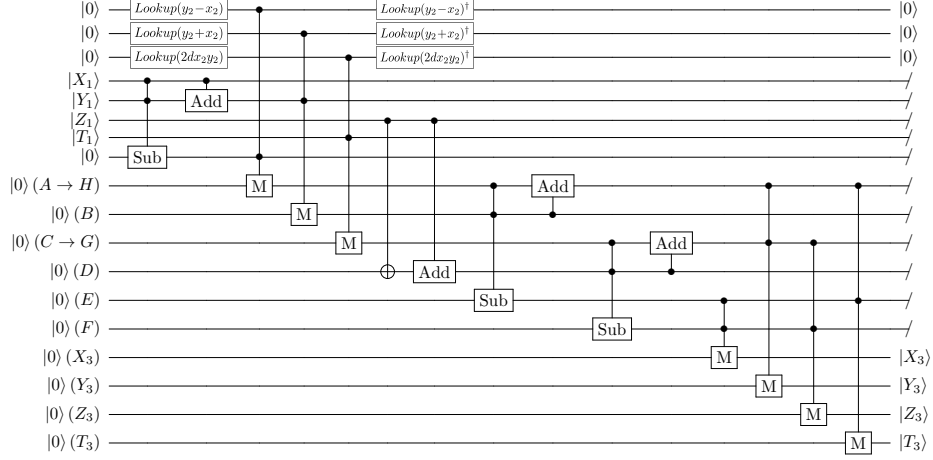


Figure 11: Point addition on Ed25519 (out-of-place, based on Section 4.4)

Table 4: Quantum resources required for point addition on Ed25519.

n	Source	Total gates	#Qubit (M)	T -depth (TD)	Full depth (FD)	$TD-M$	$FD-M$
255	This paper {	Section 4.1	$2^{28.07}$	7038	144717	$2^{29.92}$	$2^{33.84}$
		Section 4.4	$2^{25.06}$	64947	3577	$2^{27.79}$	$2^{31.36}$

5 Reimplementation of HJNRS (PQCrypto’20) ECC

In the ECC implementations by Häner, Jaques, Naehrig, Roetteler and Soeken in PQCrypto’20 [HJN⁺20], the quantum circuits and resource estimations were carried out using Microsoft’s quantum programming tool, Q# [Mic20]. As noted in their open-source project¹⁰ and in the second author’s thesis [Jaq23], the version of Microsoft Q# available at that time had issues with lower bound estimation (which have since been addressed in subsequent releases). In this part, we provide an in-depth analysis of [HJN⁺20] and report the correct quantum resource requirements by porting their quantum circuits to ProjectQ [SHT18]. In our analysis, the most critical errors arise from unrealizable parallelization of

¹⁰ <https://github.com/microsoft/QuantumEllipticCurves>

circuit operations (e.g., gates that share an ancilla qubit are reported as parallel) and from qubits that are ignored in the resource counting. We reimplement the core modules, following their algorithmic structure while keeping the depth and qubit usage within a reasonable range.

Remark 1. The lower bound estimation issues are observed in the depth-optimized version of [HJN⁺20] rather than in the qubit-optimized version. The author of [Ja23] also points out that this issue is evident in the depth-optimized version, whereas it is negligible in the qubit-optimized version.

Controlled Addition. The qubit-optimized controlled addition in [HJN⁺20] uses a single control qubit. In contrast, the depth-optimized version parallelizes the inner controlled operations using additional qubits. In our analysis, these qubits are not counted accurately due to the lower bound estimation issue. Consequently, we reimplement the controlled addition using the correct number of additional qubits.

Multiplication. In our reimplementation, following the methodology of [HJN⁺20], the general multiplication is implemented as a sequence of controlled additions, and the Montgomery reduction is implemented using quantum lookup tables. The first lookup writes the lookup value indexed by the address register (which is then added using an uncontrolled addition), followed by the second lookup to uncompute the written data. For the quantum lookup table, the method of Gidney [Gid19] is used, and we reimplement it by following the source code provided in [HJN⁺20]. In the depth-optimized version, logical AND and AND[†] [Jon13] are used for address comparison, and the data writing step is implemented using fan-out with ancilla qubits.

Inversion. As described in Section 3.3, the first n iterations of the inversion are executed unconditionally in [HJN⁺20], while the remaining n iterations include a check for whether $v = 0$ (serving as a flag; see Algorithm 2). If $v = 0$, the flag disables the round operation. After the condition $v = 0$ is reached, a counter register operates in counter mode¹¹. It is implemented as an in-place incrementer (i.e., $x \rightarrow x + 1$) whose control is the flag. The counter register is used to correct the pseudo-inverse (corresponding to k in Section 3.3).

Point Addition. Using the reimplemented modules by us, we reconstruct the point addition circuit shown in [HJN⁺20, Figure 9]. In other words, the modules in their point addition circuit that are affected by the lower-bound estimation issue are replaced with our reimplementations.

¹¹ We refer the reader to [RNSL17, Figure 9], as the counter mode is not illustrated in [HJN⁺20, Figure 6].

5.1 Benchmarking

Throughout this paper, the reported results corresponding to [HJN⁺20] (Tables 1, 2, 3, and 5) are derived from our reimplementations. We adopt the same AND gates and Toffoli gate decomposition as in [HJN⁺20] for a fair comparison. As discussed in Section 5, the lower bound estimation issue appears not to be significant for the qubit-optimized version in [HJN⁺20]. Nevertheless, we also reimplement this version (although it is not described in detail here) for a strict comparison. Notably, the required quantum resources for the qubit-optimized version are reduced in our reimplementation compared to the original report in [HJN⁺20].

6 Results

This section presents quantum resource estimates for solving the ECDLP using Shor’s algorithm. Our quantum circuits up to point addition are implemented and validated using the quantum programming framework ProjectQ [SHT18] with the *ClassicalSimulator*, and the required quantum resources are estimated using the *ResourceCounter*. The overall quantum attack costs, including Shor’s algorithm and physical costs, are then derived theoretically.

6.1 Resource Estimates for Shor’s Algorithm

In Shor’s circuit (Figure 1), $2n+2$ point additions are performed. As mentioned in Section 4.2, the windowing technique reduces this number to $2 \cdot \lceil \frac{n+1}{w} \rceil$, where w is the window size. Our out-of-place point addition is developed with windowing in mind, whereas our in-place point addition is not. To support windowing, a slight modification is made as follows. A single point addition involves three signed lookups for the point $P_2(x_2, y_2)$ (i.e., (x_2, y_2) , $3x_2$, and $(q \cdot x_2 + \bar{q} \cdot 2x_2, y_2)$) and three reverse lookups to reuse ancilla qubits, for a total of six lookups. Following [HJN⁺20, Figure 10], we employ signed point addition to handle negative points (i.e., $-P = (x, -y)$).

Windowing Size. The window size w determines a trade-off between fewer point additions and additional Toffoli gates for quantum lookup tables (in our implementation, Toffoli gates are replaced with AND gates). For example, with $w = 3$, windowed point addition reduces the number of point additions to $2 \cdot \lceil (n+1)/3 \rceil$, and each of our in-place point additions requires six lookup tables, incurring $(2^2 - 1) = 3$ AND gates per table. In this case, increasing w reduces the number of point additions more significantly than it increases the lookup table cost. Window sizes of $w = 11$ – 13 are optimal for our implementations; we omit the detailed derivation. Table 5 shows the logical quantum resources required for Shor’s algorithm with windowing for these window sizes. We can see from Table 5 that our method reduces the gate count and circuit depth, but this comes at the cost of a much higher qubit count. In this trade-off, the qubit count – T -depth and qubit count – full depth products (TD - M and FD - M) show that the circuit depth is effectively reduced with our strategy.

Table 5: Comparison of the logical quantum resources required for Shor’s attack on elliptic curves with windowing technique.

n	Source	Window size (w)	Total gates*	#Qubit (M)	T -depth (TD)	Full depth (FD)	TD - M^*	FD - M^*
192	H+[HJN+20] This paper	In-place (Q opt.)	17	$2^{32.85}$	1682	408701490	$2^{39.33}$	$2^{41.35}$
		(D opt.)	12	$2^{33.68}$	3098	15409020	$2^{35.48}$	$2^{39.07}$
		In-place	11	$2^{32.50}$ (56%)	5612	4035132	$2^{34.40}$ (52%)	$2^{38.32}$ (41%)
		Out-of-place	11	$2^{31.14}$ (83%)	87283	340041	$2^{34.79}$ (38%)	$2^{38.01}$ (52%)
224	H+[HJN+20] This paper	In-place (Q opt.)	17	$2^{33.35}$	1958	648066474	$2^{40.21}$	$2^{41.97}$
		(D opt.)	13	$2^{34.32}$	3595	20815620	$2^{36.13}$	$2^{39.73}$
		In-place	11	$2^{33.10}$ (57%)	6530	5425776	$2^{35.05}$ (53%)	$2^{38.89}$ (44%)
		Out-of-place	11	$2^{31.69}$ (84%)	103902	465468	$2^{35.49}$ (36%)	$2^{38.70}$ (51%)
256	H+[HJN+20] This paper	In-place (Q opt.)	17	$2^{33.80}$	2214	966847778	$2^{40.97}$	$2^{42.65}$
		(D opt.)	14	$2^{34.80}$	4093	26974110	$2^{36.69}$	$2^{40.35}$
		In-place	12	$2^{33.57}$ (57%)	7454	7205639	$2^{35.65}$ (51%)	$2^{39.48}$ (45%)
		Out-of-place	11	$2^{32.24}$ (83%)	129050	577344	$2^{36.12}$ (33%)	$2^{39.25}$ (53%)
384	H+[HJN+20] This paper	In-place (Q opt.)	18	$2^{35.73}$	3312	3003979914	$2^{43.18}$	$2^{44.77}$
		(D opt.)	14	$2^{36.55}$	6084	61587350	$2^{38.45}$	$2^{42.04}$
		In-place	13	$2^{35.26}$ (59%)	11133	15645480	$2^{37.35}$ (53%)	$2^{41.37}$ (37%)
		Out-of-place	12	$2^{33.88}$ (84%)	232024	1013156	$2^{37.77}$ (38%)	$2^{41.23}$ (43%)
521	H+[HJN+20] This paper	In-place (Q opt.)	19	$2^{37.34}$	4460	7070126030	$2^{44.85}$	$2^{46.60}$
		(D opt.)	15	$2^{37.82}$	8211	114902620	$2^{39.78}$	$2^{43.43}$
		In-place	13	$2^{36.64}$ (56%)	14877	30027105	$2^{38.71}$ (52%)	$2^{42.78}$ (36%)
		Out-of-place	12	$2^{35.05}$ (85%)	367827	1587862	$2^{39.09}$ (38%)	$2^{42.56}$ (45%)
255	This paper	Section 4.1	12	$2^{33.54}$	7050	6750957	$2^{35.48}$	$2^{39.46}$
		Section 4.4	11	$2^{31.67}$	101482	355578	$2^{35.07}$	$2^{38.66}$

*: Improvement percentages are relative to the D opt. results in [HJN+20].

6.2 Fault-Tolerant Estimation: Physical Qubits and Runtime

In this section, we estimate the physical costs of fault-tolerant computation, including the number of physical qubits and the runtime. We consider two error-correction codes: surface codes [FMMC12] and quantum low-density parity-check (QLDPC) codes [WSC25]. For the explanation of the resource estimation procedure, we focus on ECC-224 and assume a physical error rate of $p = 10^{-3}$, a code cycle time of $t_c = 1 \mu\text{s}$, a per-gate error rate of $p_g = 10^{-4}$, and a reaction time of $t_r = 10 \mu\text{s}$.

Surface Codes. Our estimation using surface codes follows the methodology used in [ADMG+17,BDW+25], where T gates incur a high cost. In general, a T gate on a surface code is implemented by *magic state distillation*, and we adopt the Reed-Muller 15-to-1 distillation method presented in [BK05].

Following Algorithm 5, we obtain two distillation layers with code distances $\{d_1, d_2\} = \{13, 7\}$ for ECC-224 (in-place version). The output error rate should satisfy $p_{out} = 1/T_c$, where the total number of T gates is $T_c = 2406344612$. With two distillation layers, generating a single magic state requires $N_{in} = 16 \times 15 = 240$ input states. The number of physical qubits required to encode one logical qubit at code distance d is

$$Q_{phys}(d) = 2.5 \times 1.25 \times d^2.$$

The total footprint of the bottom distillation layer (using code distance d_2) is therefore

$$Q_{bot} = N_{in} \times Q_{phys}(d_2) = 240 \times 2.5 \times 1.25 \times 7^2 = 36750$$

physical qubits, and one round of distillation completes in $C_{bot} = 10 \cdot d_2 = 70$ surface code cycles. Similarly, the top distillation layer (using code distance d_1) requires $Q_{top} = 16 \times Q_{phys}(d_1) = 16 \times 2.5 \times 1.25 \times 13^2 \approx 8450$ physical qubits and $C_{top} = 10 \cdot d_1 = 130$ surface code cycles. Since the physical qubits of the bottom layer can be reused in the top layer, the total number of physical qubits required for distillation remains $Q_{bot} = 36750$. The number of magic states produced in parallel by a single factory is

$$N_{par} = \left\lfloor \frac{Q_{bot}}{Q_{top}} \right\rfloor = \left\lfloor \frac{36750}{8450} \right\rfloor = 4.$$

A single magic state distillation takes $C_{dist} = C_{bot} + C_{top} = 200$ surface code cycles. The total time to generate the magic states for T_c T gates using one factory is:

$$T_{single} = \frac{T_c}{N_{par}} \times C_{dist} \times t_c = \frac{2406344612}{4} \times 200 \times 1 \mu s \approx 34 \text{ hours.}$$

To reduce this time, we parallelize the distillation using multiple factories. The ratio between the required number of T gates and the T -depth (based on Table 5) is $T_c/TD \approx 444$, meaning that 444 magic states must be supplied within C_{dist} cycles. The required number of distillation factories is

$$N_{fact} = \left\lceil \frac{T_c/TD}{N_{par}} \right\rceil = \left\lceil \frac{444}{4} \right\rceil = 111.$$

This increases the total number of physical qubits for distillation to $Q_{dist} = Q_{bot} \times N_{fact} = 36750 \times 111 = 4079250$, while reducing the distillation time to

$$T_{total} = \frac{T_c}{N_{par} \times N_{fact}} \times C_{dist} \times t_c = \frac{2406344612}{444} \times 200 \times 1 \mu s \approx 18 \text{ minutes.}$$

We also estimate the physical overhead for Clifford gates in the surface code. For ECC-224, the logical qubit count is $N_{log} = 6530$ and the total number of Clifford gates is $C_g = 4182005448$. The required code distance d_c is the smallest integer satisfying

$$\left(\frac{p}{0.0125} \right)^{\frac{d_c+1}{2}} < \frac{1}{C_g}.$$

Substituting $p = 10^{-3}$ and $C_g = 4182005448$, we obtain $d_c = 17$. The physical qubit count required for Clifford gates is $Q_{cliff} = N_{log} \times Q_{phys}(d_c) = 6530 \times 2.5 \times 1.25 \times 17^2 \approx 5897407$, yielding a total of $Q_{total} = Q_{dist} + Q_{cliff} = 4079250 + 5897407 = 9976657$ physical qubits for Shor's algorithm. Note that the Clifford

Algorithm 5: Procedure for calculating the number of distillation layers and code distances (as restated in [ADMG⁺17], based on [FDJ13]).

```

1: Input:  $p, p_g (= p/10), p_{out} (= 1/T_c), \varepsilon (= 1)$ 
2:  $d \leftarrow$  empty list  $[], i \leftarrow 0$ 
3: repeat
4:    $i \leftarrow i + 1, p_i \leftarrow p_{out}$ 
5:   Find minimum  $d_i$  such that  $192d_i (100p_g)^{\frac{d_i+1}{2}} < \frac{\varepsilon p_i}{1+\varepsilon}$ 
6:    $p_{out} \leftarrow \sqrt[3]{p_i / (35(1+\varepsilon))}$ 
7:    $d.append(d_i)$ 
8: until  $p_{out} > p$ 
9: Output:  $d = [d_1, \dots, d_i]$ 

```

gates do not increase the overall runtime, as they can be executed within the magic state distillation time.

The estimated physical costs of a quantum attack on ECC are summarized in Table 6. For brevity, we omit detailed estimations for other parameters, and they are derived in the same manner as for ECC-224. With a code cycle time of $1 \mu s$, breaking ECC-224 with Shor’s algorithm requires ≈ 18.1 minutes (in-place, 9976657 physical qubits) or ≈ 1.4 minutes (out-of-place, 106904493 physical qubits).

Table 6: Physical resource estimates for Shor’s algorithm using surface codes.

n	Source	Physical qubits	Runtime for code cycle time				
			$1\ \mu\text{s}$	$10\ \mu\text{s}$	$100\ \mu\text{s}$	$1\ \text{ms}$	
192	This paper {	In-place	7934838	12.8 mins	2.1 hours	21.3 hours	8.9 days
		Out-of-place	81841400	1.0 mins	10.2 mins	1.7 hours	17.0 hours
224	This paper {	In-place	9976657	18.1 mins	3.0 hours	1.3 days	12.6 days
		Out-of-place	106904493	1.4 mins	14.0 mins	2.3 hours	23.3 hours
256	This paper {	In-place	11699925	23.9 mins	4.0 hours	1.7 days	16.7 days
		Out-of-place	132316281	1.7 mins	17.3 mins	2.9 hours	1.2 days
384	This paper {	In-place	18696666	52.1 mins	8.7 hours	3.6 days	1.2 months
		Out-of-place	263074800	3.4 mins	33.7 mins	5.6 hours	2.3 days
521	This paper {	In-place	27991866	1.7 hours	17.5 hours	7.3 days	2.4 months
		Out-of-place	455637084	5.3 mins	52.9 mins	8.8 hours	3.7 days
255	This paper {	Section 4.1	11474625	22.5 mins	3.8 hours	1.6 days	15.6 days
		Section 4.4	108228931	1.1 mins	10.7 mins	1.8 hours	17.8 hours

QLDPC Codes. To estimate the physical cost using QLDPC codes [WSC25], we follow the resource model in [WBC⁺26]. Under the assumed physical setting, [WBC⁺26] uses the $d = 24$ GB-code instantiation, which encodes 16 logical

qubits per processing block with $n_{pb} = 1620$ physical qubits. The time per logical cycle is $t_\ell = d_t t_c$ with $d_t = 26$, and one magic engine is attached to each processing unit, requiring $n_{me} = 8694$ physical qubits. For the physical error rate $p = 10^{-3}$ setting, the magic-engine rejection rate is taken as $p_r = 0.06$.

The resource accounting in [WBC⁺26] counts both T -state injections and logical measurements in the total number of logical cycles. Accordingly, we estimate the total number of logical cycles by directly summing the T gate count and the measurement gate count. For our implementation of ECC-224 (in-place version), the number of logical qubits is $N = 6537$, the T gate count is $N_T = 1545312100$, and the number of measurement gates is $N_M = 294665500$. Hence, the total number of logical cycles is estimated as $L = N_T + N_M = 1839977600$. To estimate the physical qubit cost, we use the same single-processing-unit expression as in [WBC⁺26]:

$$Q_{\text{phys}} = n_{pb} \left\lceil \frac{N}{k} \right\rceil + n_{me} = 1620 \left\lceil \frac{6537}{16} \right\rceil + 8694 = 671274.$$

Under this error-correction model, our circuit is estimated to require 671274 physical qubits.

For the runtime, we account for the nonzero rejection rate of the magic engine, so logical cycles requiring T -state injections incur additional overhead. While [WBC⁺26] uses the approximation that this proportion is $2/3$ for their RSA compilation, we use the actual ratio N_T/L in our case. Therefore, we estimate the corrected number of logical cycles as

$$L_{\text{corr}} = \left(\frac{N_T}{L} (1 - p_r)^{-1} + \frac{N_M}{L} \right) L = \frac{N_T}{1 - p_r} + N_M.$$

Using $N_T = 1545312100$, $N_M = 294665500$, and $p_r = 0.06$, we obtain: $L_{\text{corr}} \approx 1.05 L$. The runtime for one shot is then estimated as $t_{\text{shot}} = L_{\text{corr}} d_t t_c$. Substituting $d_t = 26$ and $t_c = 1 \mu\text{s}$, we obtain $t_{\text{shot}} \approx 50404$ seconds ≈ 14 hours. To account for shot repetition under logical errors, we report the expected runtime $t_{\text{exp}} = t_{\text{shot}}/p_S$, where $p_S = (1 - p_L)^{N_{\text{corr}}} (1 - p_T)^{N_T}$ is the one-shot logical success probability [WBC⁺26, Eq. (30)] with $p_L \approx 4 \times 10^{-16}$ and $p_T \approx 10^{-11}$. For ECC-224 in-place, $p_S \approx 0.98$, giving $t_{\text{exp}} \approx 14.3$ hours.

Table 7 shows the estimated physical costs for a quantum attack on ECC. The Pinnacle architecture in [WBC⁺26] admits a space-time trade-off: a shorter runtime can be achieved by increasing the number of processing units, at the cost of additional physical qubits (see [WBC⁺26, Table VI]). Note that Table 7 reports a baseline estimate without parallelization. Under the resource model we adopted from [WBC⁺26], the runtime scales primarily with the T count rather than the T -depth (thus, for Table 7, we adjust w to minimize the T gate count). In this context, although our circuits are mainly optimized from the T -depth perspective, this advantage is not fully reflected in our QLDPC estimates. However, we note that recent efforts to increase parallelism by processing multiple magic states concurrently within a single QLDPC code block [LXJO⁺26] may alter this picture in future architectures.

Table 7: Physical resource estimates for Shor’s algorithm using QLDPC codes.

n	Source	Physical qubits	Runtime for code cycle time				
			1 μ s	10 μ s	100 μ s	1 ms	
192	This paper {	In-place	578934	9.4 hours	3.9 days	1.3 months	1.1 years
		Out-of-place	8497494	3.3 hours	1.4 days	13.6 days	4.5 months
224	This paper {	In-place	671274	14.3 hours	6.0 days	2.0 months	1.6 years
		Out-of-place	9712494	4.8 hours	2.0 days	20.2 days	6.7 months
256	This paper {	In-place	765234	21.9 hours	9.1 days	3.0 months	2.5 years
		Out-of-place	12142494	7.2 hours	3.0 days	1.0 months	10.0 months
384	This paper {	In-place	1137834	3.1 days	1.0 month	10.3 months	8.5 years
		Out-of-place	22802094	1.2 days	11.5 days	3.8 months	3.2 years
521	This paper {	In-place	1516914	9.5 days	3.2 months	2.6 years	26.0 years
		Out-of-place	34453134	4.5 days	1.5 months	1.2 years	12.4 years
255	This paper {	Section 4.1	724734	20.3 hours	8.4 days	2.8 months	2.3 years
		Section 4.4	10284354	6.5 hours	2.7 days	26.9 days	9.0 months

Attack Scenarios. Stewart et al. [SIZ⁺18] distinguish two quantum adversary models for ECC-based blockchain systems by attack speed. A *fast adversary* can perform live transaction hijacking, deriving the private key from a public key revealed in a pending transaction within the window between broadcast and block confirmation (e.g., ~ 10 minutes for Bitcoin), which is identified as the most severe threat [ABL⁺17]. A *slow adversary* cannot win this race but can still compromise any public key already exposed on-chain, such as those in P2PK outputs or via address reuse. To our knowledge, only our out-of-place implementation under the surface code (with a 1 μ s cycle time) qualifies as a fast adversary, though at the cost of a large number of physical qubits (see Table 6). The remaining configurations, with runtimes ranging from days to years, correspond to slow adversaries that target vulnerable P2PK outputs and reused addresses.

6.3 NIST Post-Quantum Security

NIST defines five security levels [NIS16,NIS22] with corresponding resource requirements to evaluate resilience against quantum attacks (based on logical estimation). Level 1 (corresponding to AES-128) requires that the product of the total number of gates and the full depth (G - FD) exceeds 2^{157} . However, as shown in Table 8 (derived from Table 5), the G - FD for breaking ECC is well below this threshold. This indicates that prime elliptic curves do not achieve post-quantum security as defined by NIST. In addition, NIST specifies criteria for quantum attacks in terms of the maximum circuit depth, MAXDEPTH, which is defined by a lower bound of 2^{40} and an upper bound of 2^{96} . Notably, the FD required for breaking prime field ECC does not exceed the lower bound of 2^{40} , indicating that our attack stays within the most conservative MAXDEPTH limit.

Table 8: Quantum resources and post-quantum security for Shor’s algorithm on ECC.

n	Source	#Qubit (M)	Total Gates (G)	T -depth (TD)	Full depth (FD)	Cost ($G \cdot FD$)	MAXDEPTH	NIST security*
192	This paper { In-place Out-of-place	$2^{12.45}$ $2^{16.41}$	$2^{32.50}$ $2^{31.14}$	$2^{21.94}$ $2^{18.38}$	$2^{25.86}$ $2^{21.59}$	$2^{58.36}$ $2^{52.73}$	$\checkmark$ ($< 2^{40}$)	$\times$ ($< 2^{157}$)
224	This paper { In-place Out-of-place	$2^{12.67}$ $2^{16.66}$	$2^{33.10}$ $2^{31.69}$	$2^{22.37}$ $2^{18.83}$	$2^{26.22}$ $2^{22.03}$	$2^{59.32}$ $2^{53.72}$		
256	This paper { In-place Out-of-place	$2^{12.86}$ $2^{16.98}$	$2^{33.57}$ $2^{32.24}$	$2^{22.78}$ $2^{19.14}$	$2^{26.61}$ $2^{22.27}$	$2^{60.18}$ $2^{54.51}$		
384	This paper { In-place Out-of-place	$2^{13.44}$ $2^{17.82}$	$2^{35.26}$ $2^{33.88}$	$2^{23.90}$ $2^{19.95}$	$2^{27.92}$ $2^{23.41}$	$2^{63.18}$ $2^{57.29}$		
521	This paper { In-place Out-of-place	$2^{13.86}$ $2^{18.49}$	$2^{36.64}$ $2^{35.05}$	$2^{24.84}$ $2^{20.60}$	$2^{28.92}$ $2^{24.07}$	$2^{65.56}$ $2^{59.13}$		
255	This paper { Section 4.1 Section 4.4	$2^{12.78}$ $2^{16.63}$	$2^{33.54}$ $2^{31.67}$	$2^{22.69}$ $2^{18.44}$	$2^{26.67}$ $2^{22.03}$	$2^{60.21}$ $2^{53.70}$	$\checkmark$	$\times$

*: Level 1 requires $G \cdot FD \geq 2^{157}$

7 Conclusion

As practical quantum computing continues to advance, a natural question emerges: *How difficult will it truly be to break elliptic curve cryptography?* This paper sheds light on this question by presenting new results on the quantum cryptanalysis of elliptic curves. We reduce the gate count and circuit depth required to break elliptic curves, at the cost of a much higher number of qubits. In this trade-off, our results reduce the qubit count – circuit depth product while also reducing the total gate count (as shown in Tables 3 and 5).

Based on our quantum circuits, we estimate the quantum resources required to break elliptic curves using Shor’s algorithm with windowing (in Table 5). Further, to bridge the gap to practical scenarios (inspired from [GE21, Gid25]), we evaluate our results at the fault-tolerant level and report the physical qubit requirements and runtime needed to break ECC (see Table 6).

Very recently, [BZG⁺26] reported that the 256-bit curve secp256k1 can be broken using either 1200 logical qubits and 90 million Toffoli gates, or 1450 logical qubits and 70 million Toffoli gates. Their results are better than ours in terms of qubit count and Toffoli gate count. For the comparable 256-bit curve (P-256), we require either 7454 logical qubits and 1.39 billion Toffoli gates, or 129050 logical qubits and 490 million Toffoli gates (see Table 9). Unfortunately, they do not report circuit depth, but we expect that our circuits achieve lower Toffoli depth and full depth (note that their circuit details are not disclosed due to responsible disclosure). In this context, minimizing the qubit count – Toffoli count product, which is a major consideration in [BZG⁺26] (and differs from what we focus on in this work), is a promising direction for future work.

In Gidney’s recent work [Gid25], optimizations such as approximate residue arithmetic (based on [CFS25]), yoked surface codes [GNBJ25], and Ekerå-Håstad’s period finding [EH17] further reduce quantum costs. Adapting these techniques to break elliptic curves would be an interesting direction for future work.

References

- ABL⁺17. Divesh Aggarwal, Gavin K Brennen, Troy Lee, Miklos Santha, and Marco Tomamichel. Quantum attacks on bitcoin, and how to protect against them. *arXiv preprint arXiv:1710.10377*, 2017. 28
- ADMG⁺17. Matthew Amy, Olivia Di Matteo, Vlad Gheorghiu, Michele Mosca, Alex Parent, and John Schanck. Estimating the cost of generic quantum preimage attacks on SHA-2 and SHA-3. In Roberto Avanzi and Howard Heys, editors, *Selected Areas in Cryptography – SAC 2016*, pages 317–337, Cham, 2017. Springer International Publishing. 24, 26
- AMM⁺13. Matthew Amy, Dmitri Maslov, Michele Mosca, Martin Roetteler, and Martin Roetteler. A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 32(6):818–830, Jun 2013. 7
- BBvHL20. Gustavo Banegas, Daniel J. Bernstein, Iggy van Hoof, and Tanja Lange. Concrete quantum cryptanalysis of binary elliptic curves. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021:451–472, 12 2020. 2
- BDL⁺12. Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of cryptographic engineering*, 2(2):77–89, 2012. 19
- BDW⁺25. Anik Basu Bhaumik, Suman Dutta, Siyi Wang, Anubhab Baksi, Kyungbae Jang, Amit Saha, Hwajeong Seo, and Anupam Chattopadhyay. Can quantum break zuc? only with a million qubits and a billion years to spare. *Cryptology ePrint Archive*, 2025. 24
- BK05. Sergey Bravyi and Alexei Kitaev. Universal quantum computation with ideal clifford gates and noisy ancillas. *Physical Review A—Atomic, Molecular, and Optical Physics*, 71(2):022316, 2005. 24
- Bro10. Daniel RL Brown. Sec 2: Recommended elliptic curve domain parameters. *Standards for Efficient Cryptography*, 2010. 2
- BZG⁺26. Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations. *arXiv preprint arXiv:2603.28846*, 2026. 2, 4, 29, 42
- CFS25. Clémence Chevnard, Pierre-Alain Fouque, and André Schrottenloher. Reducing the number of qubits in quantum factoring. In *Annual International Cryptology Conference*, pages 384–415. Springer, 2025. 29
- CFS26. Clémence Chevnard, Pierre-Alain Fouque, and André Schrottenloher. Reducing the number of qubits in quantum discrete logarithms on elliptic curves. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 371–401. Springer, 2026. 2, 4, 17, 42
- CMR⁺23. Lily Chen, Dustin Moody, Karen Randall, Andrew Regenscheid, and Angela Robinson. Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters nist special publication nist sp 800-186, February 2023. 36
- DKRS04. Thomas G Draper, Samuel A Kutin, Eric M Rains, and Krysta M Svore. A logarithmic-depth quantum carry-lookahead adder. *arXiv preprint quant-ph/0406142*, 2004. 8, 10

- EH17. Martin Ekerå and Johan Håstad. Quantum algorithms for computing short discrete logarithms and factoring rsa integers. In *Post-Quantum Cryptography: 8th International Workshop, PQCrypto 2017, Utrecht, The Netherlands, June 26-28, 2017, Proceedings 8*, pages 347–363. Springer, 2017. 29
- Eke19. Martin Ekerå. Revisiting shor’s quantum algorithm for computing general discrete logarithms. *arXiv preprint arXiv:1905.09084*, 2019. 6
- FDJ13. Austin G Fowler, Simon J Devitt, and Cody Jones. Surface code implementation of block code state distillation. *Scientific reports*, 3(1):1939, 2013. 26
- FMMC12. Austin G Fowler, Matteo Mariantoni, John M Martinis, and Andrew N Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A—Atomic, Molecular, and Optical Physics*, 86(3):032324, 2012. 4, 24
- GE21. Craig Gidney and Martin Ekerå. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. 18, 29
- Gid18. Craig Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, 2018. 7
- Gid19. Craig Gidney. Windowed quantum arithmetic. *arXiv preprint arXiv:1905.07682*, 2019. 22
- Gid25. Craig Gidney. How to factor 2048 bit rsa integers with less than a million noisy qubits. *arXiv preprint arXiv:2505.15917*, 2025. 29
- GLRS16. Markus Grassl, Brandon Langenberg, Martin Roetteler, and Rainer Steinwandt. Applying Grover’s algorithm to AES: Quantum resource estimates. In Tsuyoshi Takagi, editor, *Post-Quantum Cryptography*, pages 29–43, Cham, 2016. Springer International Publishing. 18
- GN96. Robert B Griffiths and Chi-Sheng Niu. Semiclassical fourier transform for quantum computation. *Physical Review Letters*, 76(17):3228, 1996. 6, 41
- GNBJ25. Craig Gidney, Michael Newman, Peter Brooks, and Cody Jones. Yoked surface codes. *Nature Communications*, 16(1):4498, 2025. 29
- GRLR⁺23. Élie Gouzien, Diego Ruiz, Francois-Marie Le Régent, Jérémie Guillaud, and Nicolas Sangouard. Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126 133 cat qubits. *Physical review letters*, 131(4):040602, 2023. 2, 12
- HJN⁺20. Thomas Häner, Samuel Jaques, Michael Naehrig, Martin Roetteler, and Mathias Soeken. Improved quantum circuits for elliptic curve discrete logarithms. In *Post-Quantum Cryptography: 11th International Conference, PQCrypto 2020, Paris, France, April 15–17, 2020, Proceedings 11*, pages 425–444. Springer, 2020. 2, 3, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 41, 42, 43
- HZH⁺25. Yan Huang, Fangguo Zhang, Zhi Hu, Zijian Zhou, and Longjiang Qu. Improved quantum circuits for elliptic curve discrete logarithm problems on ed25519: Y. huang et al. *Quantum Information Processing*, 24(10):311, 2025. 2
- Jaq23. Samuel Jaques. *The do’s and don’ts of quantum memory in cryptanalysis*. PhD thesis, University of Oxford, 2023. 21, 22
- JNRV19. Samuel Jaques, Michael Naehrig, Martin Roetteler, and Fernando Virdia. Implementing grover oracles for quantum key search on AES and LowMC. Cryptology ePrint Archive, Report 2019/1146, 2019. <https://eprint.iacr.org/2019/1146>. 7, 8

- JNRV20. Samuel Jaques, Michael Naehrig, Martin Roetteler, and Fernando Vir-
dia. Implementing grover oracles for quantum key search on AES and
lowmc. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptol-
ogy - EUROCRYPT 2020 - 39th Annual International Conference on the
Theory and Applications of Cryptographic Techniques, Zagreb, Croatia,
May 10-14, 2020, Proceedings, Part II*, volume 12106 of *Lecture Notes in
Computer Science*, pages 280–310. Springer, 2020. 3
- Jon13. Cody Jones. Low-overhead constructions for the fault-tolerant tof-
foli gate. *Physical Review A—Atomic, Molecular, and Optical Physics*,
87(2):022328, 2013. 22
- JSB⁺25. Kyungbae Jang, Vikas Srivastava, Anubhab Baksi, Santanu Sarkar, and
Hwajeong Seo. New quantum cryptanalysis of binary elliptic curves.
IACR Transactions on Cryptographic Hardware and Embedded Systems,
2025(2):781–804, 2025. 2, 3, 17
- Kal02. Burton S Kaliski. The montgomery inverse and its applications. *IEEE
transactions on computers*, 44(8):1064–1065, 2002. 12
- KLJ⁺25. Hyunjun Kim, Sejin Lim, Kyungbae Jang, Siyi Wang, Anubhab Baksi,
Anupam Chattopadhyay, and Hwajeong Seo. Tree-based quantum carry-
save adder. *Cryptology ePrint Archive*, 2025. 3, 9
- Kob87. Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*,
48(177):203–209, 1987. 2
- LL25. Haoyu Liao and Qingbin Luo. Quantum circuit synthesis for aes with low
dw-cost. In *International Conference on the Theory and Application of
Cryptology and Information Security*, pages 475–507. Springer, 2025. 3
- LPZW23. Qun Liu, Bart Preneel, Zheng Zhao, and Meiqin Wang. Improved quan-
tum circuits for aes: reducing the depth and the number of qubits. In
*International Conference on the Theory and Application of Cryptology
and Information Security*, pages 67–98. Springer, 2023. 3
- LXJO⁺26. Kun Liu, Shifan Xu, Tomas Jochym-O’Connor, Zhiyang He, Shraddha
Singh, and Yongshan Ding. In-situ simultaneous magic state injection on
arbitrary css qldpc codes. *arXiv preprint arXiv:2604.05126*, 2026. 27
- LYW⁺26. Han Luo, Ziyi Yang, Ziruo Wang, Yuexin Su, and Tongyang Li. Space-
efficient quantum algorithm for elliptic curve discrete logarithms with
resource estimation. *arXiv preprint arXiv:2604.02311*, 2026. 2, 4, 42
- Mic20. Microsoft. *Q# Language Specification*, 2020. 21
- MS16. Dustin Moody and Daniel Shumow. Analogues of vélú’s formulas for iso-
genies on alternate models of elliptic curves. *Mathematics of Computation*,
85(300):1929–1951, 2016. 20
- NIS16. NIST. Submission requirements and evaluation criteria for the post-
quantum cryptography standardization process, 2016. [https://csrc.n
ist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/document
s/call-for-proposals-final-dec-2016.pdf](https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf). 28
- NIS22. NIST. Call for additional digital signature schemes for the post-quantum
cryptography standardization process, 2022. [https://csrc.nist.gov/cs
rc/media/Projects/pqc-dig-sig/documents/call-for-proposals-d
ig-sig-sept-2022.pdf](https://csrc.nist.gov/cs
rc/media/Projects/pqc-dig-sig/documents/call-for-proposals-d
ig-sig-sept-2022.pdf). 4, 28
- PZ03. John Proos and Christof Zalka. Shor’s discrete logarithm quantum algo-
rithm for elliptic curves. *arXiv preprint quant-ph/0301141*, 2003. 2
- RNSL17. Martin Roetteler, Michael Naehrig, Krysta M Svore, and Kristin Lauter.
Quantum resource estimates for computing elliptic curve discrete loga-
rithms. In *Advances in Cryptology—ASIACRYPT 2017: 23rd International*

- Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II 23*, pages 241–270. Springer, 2017. 2, 8, 9, 10, 11, 12, 17, 22, 41, 43
- RSA78. Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. 2
- Sel13. Peter Selinger. Quantum circuits of t-depth one. *Physical Review A—Atomic, Molecular, and Optical Physics*, 87(4):042302, 2013. 7
- Sho94. Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994. 2, 5
- SHT18. Damian S Steiger, Thomas Häner, and Matthias Troyer. Projectq: an open source software framework for quantum computing. *Quantum*, 2:49, 2018. 21, 23
- SIZ⁺18. Iain Stewart, Daniel Ilie, Alexei Zamyatin, Sam Werner, MF Torshizi, and William J Knottenbelt. Committing to quantum resistance: a slow defence for bitcoin against a fast quantum computing attack. *Royal Society open science*, 5(6), 2018. 28
- TT23. Ren Taguchi and Atsushi Takayasu. Concrete quantum cryptanalysis of binary elliptic curves via addition chain. In *Cryptographers’ Track at the RSA Conference*, pages 57–83. Springer, 2023. 2
- WBC⁺26. Paul Webster, Lucas Berent, Omprakash Chandra, Evan T Hockings, Nouédyn Baspin, Felix Thomsen, Samuel C Smith, and Lawrence Z Cohen. The pinnacle architecture: Reducing the cost of breaking rsa-2048 to 100 000 physical qubits using quantum ldpc codes. *arXiv preprint arXiv:2602.11457*, 2026. 26, 27
- WSC25. Paul Webster, Samuel C Smith, and Lawrence Z Cohen. Explicit construction of low-overhead gadgets for gates on quantum ldpc codes. *arXiv preprint arXiv:2511.15989*, 2025. 4, 24, 26
- ZWS⁺20. Jian Zou, Zihao Wei, Siwei Sun, Ximeng Liu, and Wenling Wu. Quantum circuit implementations of aes with fewer qubits. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 697–726, Cham, 2020. Springer International Publishing. 3

A Group Law for Weierstrass Curves

Let $W_{a,b}$ be a Weierstrass elliptic curve defined over the prime field $\text{GF}(p)$. The point at infinity $\mathcal{O}$ acts as the identity element for the elliptic-curve group law. Hence, for every point P on $W_{a,b}$, we have $P + \mathcal{O} = \mathcal{O} + P = P$. If $P = (x, y)$ is an affine point on $W_{a,b}$, then its inverse with respect to the group law is obtained by reflecting P across the x -axis. Thus, $-P = (x, -y)$, where the second coordinate is understood modulo p . Consequently, $P + (-P) = \mathcal{O}$.

Now let $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ be two distinct points on $W_{a,b}$ such that $P_1 \neq P_2$. If $Q = P_1 + P_2 = (x, y)$, then the coordinates of Q are given by

$$x = \lambda^2 - x_1 - x_2, \quad y = \lambda(x_1 - x) - y_1, \quad \text{where } \lambda = \frac{y_2 - y_1}{x_2 - x_1}.$$

Here the division is performed in $\text{GF}(p)$. For point doubling, let $P = (x_1, y_1)$ be a point on $W_{a,b}$ with $P \neq -P$. If $Q = 2P = (x, y)$, then

$$x = \lambda^2 - 2x_1, \quad y = \lambda(x_1 - x) - y_1, \quad \text{where } \lambda = \frac{3x_1^2 + a}{2y_1}.$$

Again, all arithmetic is carried out modulo p .

In order to avoid the costly field inversion required in affine coordinates, it is common to represent points using projective coordinates. In **Jacobian projective coordinates**, a point is represented as $P = (X : Y : Z)$. The corresponding affine point is given by $x = X/Z^2$ and $y = Y/Z^3$ for $Z \neq 0$. For two distinct finite points $P_1 = (X_1 : Y_1 : Z_1)$ and $P_2 = (X_2 : Y_2 : Z_2)$ with $P_2 \neq \pm P_1$, define

$$U_1 = X_1 Z_2^2, \quad U_2 = X_2 Z_1^2, \quad S_1 = Y_1 Z_2^3, \quad S_2 = Y_2 Z_1^3,$$

and

$$H = U_2 - U_1, \quad r = S_2 - S_1.$$

Then $P_3 = P_1 + P_2 = (X_3 : Y_3 : Z_3)$ is given by

$$X_3 = r^2 - H^3 - 2U_1 H^2, \quad Y_3 = r(U_1 H^2 - X_3) - S_1 H^3, \quad Z_3 = Z_1 Z_2 H.$$

Unlike affine coordinates, Jacobian projective coordinates avoid field division during point addition. However, they are not uniquely determined for a given point, since $(X : Y : Z)$ and $(c^2 X : c^3 Y : cZ)$ represent the same affine point for any $c \in \text{GF}(p)^*$. Similarly, for the other representations of elliptic curves over $\text{GF}(p)$, namely Montgomery curves and (twisted) Edwards curves, appropriate group laws can be defined.

B Key Establishment using Prime Elliptic Curves

The *Elliptic Curve Diffie-Hellman* (ECDH) key establishment protocol is an ECC-based anonymous key agreement protocol that allows two parties (say Alice and Bob), each possessing an elliptic curve public-private key pair, to establish a

shared secret over an insecure communication channel. Private keys are randomly chosen integer scalars, while public keys are points on the curve. The security of ECDH depends on the variant of discrete logarithm problem known as Elliptic Curve Discrete Logarithm Problem (ECDLP). The ECDLP problem is states as follows: Let E be an elliptic curve defined over a finite field $\text{GF}(q)$; and let $G, H \in E(\text{GF}(q))$ be points on the elliptic curve group such that $H \in \langle G \rangle$. The ECDLP asks to find the integer m , such that $H = [m]G$. The ECDLP is a special case of the discrete logarithm problem in which the cyclic group is represented by the group $\langle G \rangle$ of points on an elliptic curve.

At the first step, all parties must agree on all the elements defining the elliptic curve (also called, *domain parameters*) for the protocol. The prime elliptic curve is uniquely determined by the field $\text{GF}(p)$ over which the elliptic curve is defined, the parameters of its defining equation, and principal parameters, such as the cofactor h of the curve, the order n of its prime-order subgroup, and a designated point G on the curve of order n (i.e., the “base point”). Note that order of elliptic curve group is given by hn . Thus, domain parameters for prime ECC is given by (p, h, a, b, G, n) . Once the domain parameter has been decided, Alice and Bob proceed as follows. Alice randomly selects an integer sk_A from $\{2, \dots, n-1\}$, and computes $PK_A = [sk_A]G$, and sends PK_A to Bob. Similarly, Bob selects an integer sk_B from $\{2, \dots, n-1\}$, computes $PK_B = [sk_B]G$, and sends PK_B to Alice. Upon receiving PK_B and PK_A , Alice and Bob can compute the shared secret key SK independently as,

$$S = [sk_A]PK_B = [sk_B]PK_A = [sk_A][sk_B]G = [sk_A \cdot sk_B]G$$

Both Alice and Bob arrive at the same S , thereby establishing the shared key. Worked-out example of key establishment using ECC is given in Example 8.

C Worked-out Examples

C.1 Prime Fields

Example 1 (Toy Example of a Prime Field). Consider the prime field $\text{GF}(7)$, where $p = 7$ is a prime. Then

$$\text{GF}(7) = \{0, 1, 2, 3, 4, 5, 6\}.$$

Addition Table of $\text{GF}(7)$:

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

Multiplication Table of $\text{GF}(7)$:

·	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

One important observation from the addition table is that the additive inverse $(-a)$ of an element $a \in \text{GF}(7)$ is the element which sums to 0 modulo 7. For example, if $a = 2$, then

$$-2 \equiv 5 \pmod{7}, \quad \text{since } 2 + 5 \equiv 0 \pmod{7}.$$

□

Example 2 (Standard Example (adopted from [CMR⁺23]) of Prime Field). Let $p = 2^{224} - 2^{96} + 1$ be a prime number. The corresponding prime field is defined as $\text{GF}(p) = \{0, 1, 2, \dots, p-1\}$, where the field operations are performed modulo p . This prime field is used to define the elliptic curve P-224.

Example 3 (Standard Example (adopted from [CMR⁺23]) of Prime Field). Let $p = 2^{255} - 19$ be a prime number. The corresponding prime field is defined as $\text{GF}(p) = \{0, 1, 2, \dots, p-1\}$, with arithmetic performed modulo p . This field is used to define the elliptic curve Curve25519, which is a Montgomery curve over $\text{GF}(p)$.

C.2 Prime Elliptic Curves

Example 4 (Toy Example of Prime Elliptic Curve). Let $p = 17$ and consider the elliptic curve E defined over the prime field $\text{GF}(17)$ by

$$E : y^2 \equiv x^3 + 9x + 1 \pmod{17}.$$

We write $E(\text{GF}(17))$ for the set of all points on the curve over the field $\text{GF}(17)$. Thus,

$$E(\text{GF}(17)) = \{(x, y) \in \text{GF}(17)^2 : y^2 \equiv x^3 + 9x + 1 \pmod{17}\} \cup \{\mathcal{O}\},$$

where $\mathcal{O}$ denotes the point at infinity. For each fixed $x \in \text{GF}(17)$, define

$$r(x) \equiv x^3 + 9x + 1 \pmod{17}.$$

Then a point of $E(\text{GF}(17))$ with x -coordinate equal to x exists if and only if $r(x)$ is a quadratic residue modulo 17, i.e., if there exists some $y \in \text{GF}(17)$ such that

$$y^2 \equiv r(x) \pmod{17}.$$

If such a y exists and $y \neq 0$, then the two corresponding points are (x, y) and $(x, -y)$, where $-y$ is taken modulo 17. Equivalently, the second point is $(x, 17 - y)$. For example, when $x = 0$, we get

$$r(0) \equiv 0^3 + 9 \cdot 0 + 1 \equiv 1 \pmod{17}.$$

Since $1^2 \equiv 1 \pmod{17}$ and $(-1)^2 \equiv 16^2 \equiv 1 \pmod{17}$, we obtain the two points $(0, 1), (0, 16) \in E(\text{GF}(17))$. For $x = 1$, we have

$$r(1) \equiv 1^3 + 9 \cdot 1 + 1 \equiv 11 \pmod{17}.$$

However, 11 is not a quadratic residue modulo 17. Hence there is no $y \in \mathbb{F}_{17}$ such that $y^2 \equiv 11 \pmod{17}$, and therefore there is no point of $E(\text{GF}(17))$ with x -coordinate equal to 1.

For $x = 3$, we get

$$r(3) \equiv 3^3 + 9 \cdot 3 + 1 \equiv 27 + 27 + 1 \equiv 55 \equiv 4 \pmod{17}.$$

Since $2^2 \equiv 4 \pmod{17}$ and $(-2)^2 \equiv 15^2 \equiv 4 \pmod{17}$, we obtain the two points $(3, 2), (3, 15) \in E(\text{GF}(17))$. Similarly, for $x = 4$, we have $r(4) \equiv 4^3 + 9 \cdot 4 + 1 \equiv 64 + 36 + 1 \equiv 101 \equiv 16 \pmod{17}$. Since $4^2 \equiv 16 \pmod{17}$ and $(-4)^2 \equiv 13^2 \equiv 16 \pmod{17}$, we obtain the two points $(4, 4), (4, 13) \in E(\text{GF}(17))$. We can proceed in the same way for all $x \in \text{GF}(17)$.

The list of all points on the prime elliptic curve $E(\text{GF}(17))$ in affine coordinates is given next:

$$\begin{aligned} E(\text{GF}(17)) = \{ & \mathcal{O}, (0, 1), (0, 16), (3, 2), (3, 15), (4, 4), (4, 13), (5, 1), (5, 16), (6, 4), \\ & (6, 13), (7, 4), (7, 13), (12, 1), (12, 16), (14, 7), (14, 10), (15, 3), \\ & (15, 14), (16, 5), (16, 12) \}. \end{aligned}$$

Note that $|E(\text{GF}(17))| = 21$. Let's take $P = (6, 4), Q = (3, 2)$. We illustrate the process of computing the point addition. Since $P \neq Q$, we use

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}.$$

Thus, $\lambda = \frac{2-4}{3-6} = \frac{-2}{-3} = \frac{2}{3} \pmod{17}$. Since $3^{-1} \equiv 6 \pmod{17}$, we get $\lambda \equiv 2 \cdot 6 \equiv 12 \pmod{17}$. Therefore, $x = \lambda^2 - x_1 - x_2 \equiv 12^2 - 6 - 3 \equiv 16 \pmod{17}$, and $y = \lambda(x_1 - x) - y_1 \equiv 12(6 - 16) - 4 \equiv 12 \pmod{17}$. Hence, $P + Q = (16, 12)$. Next, we compute $2P$. For point doubling,

$$\lambda = \frac{3x_1^2 + a}{2y_1}.$$

Here $a = 9$ and $P = (6, 4)$, so

$$\lambda = \frac{3 \cdot 6^2 + 9}{2 \cdot 4} = \frac{108 + 9}{8} = \frac{117}{8} \pmod{17}.$$

Now, $117 \equiv 15 \pmod{17}$, and $8^{-1} \equiv 15 \pmod{17}$. Therefore, $\lambda \equiv 15 \cdot 15 \equiv 225 \equiv 4 \pmod{17}$. Thus, $x = \lambda^2 - 2x_1 \equiv 4 \pmod{17}$, and $y = \lambda(x_1 - x) - y_1 \equiv 4 \pmod{17}$. Hence, $2P = (4, 4)$. $\square$

Example 5 (Prime Elliptic Curve: Ed25519). The elliptic curve Ed25519 is the twisted Edwards curve $E_{a,d}$

$$E_{a,d} : ax^2 + y^2 = 1 + dx^2y^2,$$

defined over the prime field $\text{GF}(p)$ with $p = 2^{255} - 19$ and parameters $a = -1$ and $d = -\frac{121665}{121666}$. The size of the elliptic curve group of the curve Ed25519 is $8n$, where n is a prime number. The other parameters of Ed25519 are given below:

- $a : 115792089210356248762697446949407573530086143415290314195533631$
 308867097853948
- (= 0xffffffff00000001000000000000000000000000ffffffffffffc)
- Curve parameter:
- $b : 410583637251521421293261297800472684091144410159937255548352563$
 14039467401291
- (= 0x5ac635d8aa3a93e7b3ebbd55f69886bc651d06b0cc53b0f63bce3c3e27
d2604b)
- Generator point $G = (G_x, G_y) \in P-256$ of a subgroup of prime order n ,
where
- $G_x : 4843956129390645175905258525279791420276294952604174799584408$
 $0717082404635286.$

G_y : 3613425095674979579858512791958788195661110667298501507187719
8253568414405109
(= 0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837
bf51f5).

$$W_{a,b}: \quad y^2 \equiv x^3 + ax + b \pmod{p},$$

- Prime subgroup order:
 n : 39402006196394479212279040100143613805079739270465446667946905279627659399113263569398956308152294913554433653942643
(= 0xffc7634d81f4372ddf581a0db248b0a77aecec196accc52973)

$$a = -3.$$

– Curve parameter:
 $b : 275801935599597058778490118403890480930569058563615685214287073$
 $01988689241309860865136260764883745107765439761230575$
 $(= 0xb3312fa7e23ee7e4988e056be3f82d19181d9c6efe8141120314088f50$
 $13875ac656398d8a2ed19d2a85c8edd3ec2aef)$

G_x : 26247035095799689268623156744566981891852923491109213387815615900925518854738050089022388053975719786650872476732087,
(= 0xaa87ca22be8b05378eb1c71ef320ad746e1d3b628ba79b9859f741e082542a385502f25dbf55296c3a545e3872760ab7), and

G_y : 8325710961489029985546751289520108179287853048861315594709205
902480503199884419224438643760392947333078086511627871
(= 0x3617de4a96262c6f5d9e98bf9292dc29f8f41dbd289a147ce9da3113b5
f0b8c00a60b1ce1d7e819d7a431d7c90ea0e5f).

C.3 Key Establishment with ECC

Example 8 (Standard key establishment using ECC). Consider the standard domain parameters (p, h, a, b, G, n) for the prime elliptic curve P-224 with

- $p = 2^{224} - 2^{96} + 1$.
- $h = 1$
- $n : 26959946667150639794667015087019625940457807714424391721682722368061$
(= 0xffffffffff16a2e0b8f03e13dd29455c5c2a3d)
- Curve parameter:
 $a = -3$.

Equivalently,

- $a : 26959946667150639794667015087019630673557916260026308143510066298878$
(= 0xfffffffffffffffefffffffffffffffffffffffe)
- Curve parameter:
 $b : 18958286285566608000408668544493926415504680968679321075787234672564$
(= 0xb4050a850c04b3abf54132565044b0b7d7bfd8ba270b39432355ffb4)
- Generator point $G = (G_x, G_y) \in$ P-224 of a subgroup of prime order n , where
 $G_x : 19277929113566293071110308034699488026831934219452440156649784352033$,
(= 0xb70e0cbd6bb4bf7f321390b94a03c1d356c21122343280d6115c1d21),
and
 $G_y : 19926808758034470970197974370888749184205991990603949537637343198772$
(= 0xbd376388b5f723fb4c22dfe6cd4375a05a07476444d5819985007e34).

Note that elliptic curve is $W_{a,b} : y^2 \equiv x^3 + ax + b \pmod{p}$, defined over the prime field $\text{GF}(p)$. The size of elliptic curve group is, hn with the subgroup generated by G having size n .

- Alice's private key: $sk_A = 22960898917867061707922035829769189355108621392661500359822746413025$
- Alice's public key: $pk_A = (0x0x37389ec67952028ea8b4acbfaf92356130cef725d5dcbaa24d4e42f4, 0x0x9aa675f25c94a0a53bc38229417f1ac098add7c4e0b3245ceb4bc287)$
- Bob's private key: $sk_B = 14963080162153841562746390734434733686808642118824624623665378019114$
- Bob's public key: $pk_B = (0x63fe95437df8615fe145df1acf32d86f2852ca272db8815167ca507c, 0x206d2d351ef4e48c61bfdbbab9421e82212d11f5b096b24d90cb181b)$
- Shared secret: $(0x8cdda59c9e4a01e557911747d76c78edd24a353720978a77a46d88cf, 0xc6a60a92fc87e157923c9a9444c1d1604bee88b356b33a94a6863b3d)$.

□

D Semi-Classical Fourier Transform

Shor’s quantum circuit requires multiple control qubits along with Hadamard gates, controlled rotations, and measurements. Griffiths and Niu [GN96] showed that the quantum Fourier transform can be performed semi-classically by measuring the qubits one at a time and using the classical measurement results

$$m_0, \dots, m_{2n+1} \quad \text{with} \quad \theta_k = -\pi \sum_{j=0}^{k-1} 2^{k-j} m_j$$

to control the following operations. Since each qubit is measured right after it is used, the $2n+2$ qubits originally required for the quantum Fourier transform can be replaced by a single reusable qubit. Figure 12 shows the quantum circuit for computing elliptic curve discrete logarithms based on this semi-classical Fourier transform.

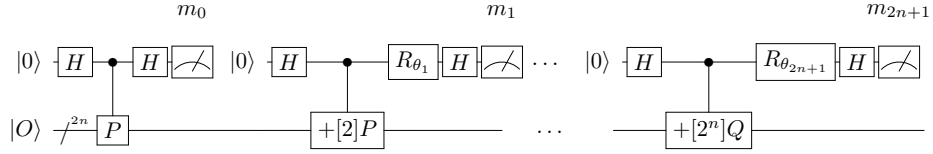


Figure 12: Quantum circuit of Shor’s algorithm for elliptic curve logarithms with a semi-classical Fourier transform.

E Quantum Circuits for Montgomery Multiplication

This part presents the quantum circuit diagrams of the Montgomery multiplication algorithms proposed in [RNSL17] and [HJN⁺20], which are described in Section 3.2 for comparison with our proposed design (Figure 5). Specifically, Figure 13(a) illustrates the circuit from [RNSL17], while Figure 13(b) shows the corresponding circuit from [HJN⁺20].

F Quantum Lookup Table vs. Quantum Circuit Synthesis

In some cases, replacing arithmetic with a quantum lookup table (as done in [HJN⁺20]) is clearly more efficient than synthesizing it into quantum circuits. As one example, there is no doubt that using a quantum lookup table to replace quantum point additions at the level of Shor’s algorithm is effective, given the high cost of quantum point additions. However, in some cases, depending on the optimization goal, we can choose either to adopt a quantum lookup table or to synthesize the operation into quantum circuits.

For the Montgomery reduction in [HJN⁺20], depending on the word size w , w quantum additions are replaced by two quantum lookup tables (including one

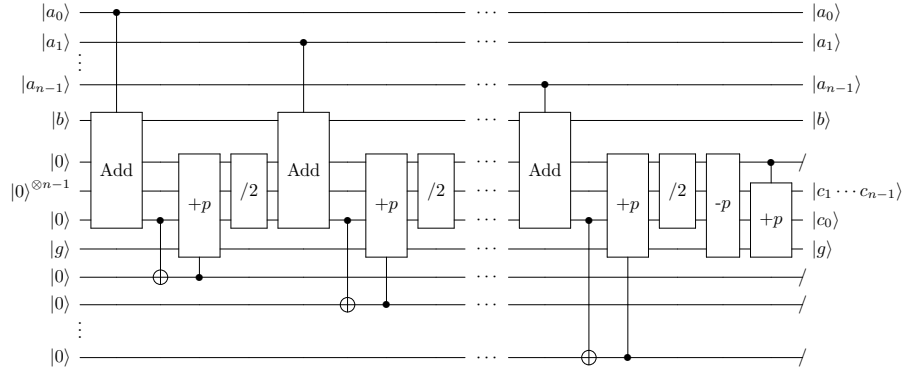
for reversal) and one quantum addition. In [HJN⁺20], for $w = 8$ and $n = 256$, the full depth of a quantum lookup table is approximately 7000. In contrast, our Montgomery reduction is designed to optimize circuit depth and is implemented using w quantum additions handled by the QCSA, achieving a full depth of 386 under the same parameters. However, this requires a much larger number of qubits.

G Comparison With CFS, LYW⁺, and BZG⁺

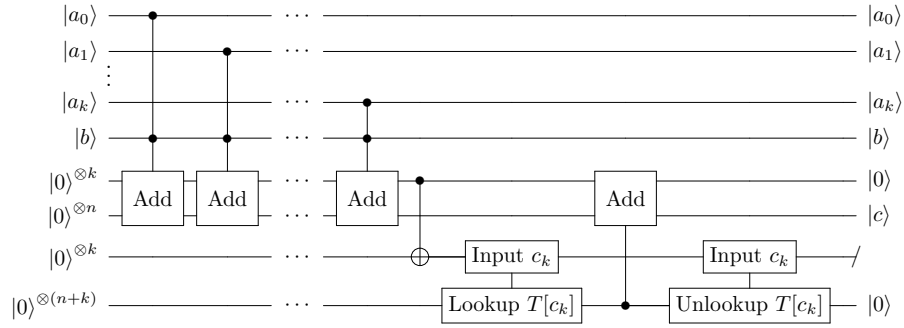
Table 9 compares our work with Chevignard et al. [CFS26], Luo et al. [LYW⁺26], and Babbush et al. [BZG⁺26]. These works report qubit and Toffoli gate counts, but not circuit depth. They use substantially fewer qubits than our implementations. Our circuits use fewer Toffoli gates than those of Chevignard et al. [CFS26] and Luo et al. [LYW⁺26], whereas Babbush et al. [BZG⁺26] report a lower count for secp256k1 than ours for P-256. A comparison of circuit depth is not possible because these works do not report T -depth or full depth. Nevertheless, given our design choices (such as the QCSA-based Montgomery multiplication in Section 3.2, the low-depth inversion in Section 3.3, and the point addition in Section 4), we expect our circuits to achieve lower T -depth and full depth.

Table 9: Quantum resources for Shor’s algorithm on elliptic curves, compared with Chevignard et al. [CFS26], Luo et al. [LYW⁺26] and Babbush et al. [BZG⁺26].

Curve	n	Source	#Qubit	Toffoli gates	T -depth	Full depth
P-224	224	Chevignard et al. [CFS26]	1098	$2^{42.42}$	-	-
		Luo et al. [LYW ⁺ 26]	1169	$2^{33.35}$	-	-
		This paper $\left\{ \begin{array}{l} \text{In-place} \\ \text{Out-of-place} \end{array} \right.$	6530	$2^{29.92}$	$2^{22.37}$	$2^{26.22}$
			103902	$2^{28.28}$	$2^{18.83}$	$2^{22.03}$
P-256	256	Chevignard et al. [CFS26]	1193	$2^{43.44}$	-	-
		Luo et al. [LYW ⁺ 26]	1333	$2^{33.93}$	-	-
		This paper $\left\{ \begin{array}{l} \text{In-place} \\ \text{Out-of-place} \end{array} \right.$	7454	$2^{30.37}$	$2^{22.78}$	$2^{26.61}$
			129050	$2^{28.87}$	$2^{19.14}$	$2^{22.27}$
P-384	384	Chevignard et al. [CFS26]	1494	$2^{46.05}$	-	-
		Luo et al. [LYW ⁺ 26]	1973	$2^{35.69}$	-	-
		This paper $\left\{ \begin{array}{l} \text{In-place} \\ \text{Out-of-place} \end{array} \right.$	11133	$2^{32.06}$	$2^{23.90}$	$2^{27.92}$
			232024	$2^{30.50}$	$2^{19.95}$	$2^{23.41}$
P-521	521	Chevignard et al. [CFS26]	1895	$2^{48.02}$	-	-
		Luo et al. [LYW ⁺ 26]	2662	$2^{37.01}$	-	-
		This paper $\left\{ \begin{array}{l} \text{In-place} \\ \text{Out-of-place} \end{array} \right.$	14877	$2^{33.49}$	$2^{24.84}$	$2^{28.92}$
			367827	$2^{31.69}$	$2^{20.60}$	$2^{24.07}$
secp256k1	256	Babbush et al. [BZG ⁺ 26]	≤ 1200	$\leq 2^{26.42}$	-	-
			≤ 1450	$\leq 2^{26.06}$	-	-



(a) Bit-level (in [RNSL17]).



(b) Word-level and windowed (in [HJN+20]).

Figure 13: Montgomery multiplication circuits.